

FIT3152 Assignment 3

Ng Li Xian 33986010

Tasks 1

This section describes the collection of a corpus comprising documents from four different genres: Music, Recipes, News, and Movie/TV Show Reviews. To ensure balanced representation across genres, five documents were selected from each category, resulting in a total of 20 documents. The documents were obtained from various online sources, including news platforms, recipe websites, music-related articles, and movie/TV review blogs. Each document contains a minimum of 200 words to ensure sufficient textual content for analysis. To acknowledge the original sources, all documents are referenced in the References section using APA 7th edition citation style.

Tasks 2

This section focuses on the data collection process. To maintain consistency, all of the text files are renamed as <genre>_<count>.txt. The table below shows the process of how data is collected and stored and is performed iteratively for all 20 documents. After all of the text files are created, they are moved into the Corpus folder.

Table 2.1 – Data Collection and Storage Process Step

Step	Description of Steps
1	Copy the related text from online sources such as articles and news
2	Paste the collected content into Microsoft Notepad
3	Modify the text by removing unimportant content such as advertisement and other non-content material
4	Save into plaintext format (.txt) and set the encoding to Unicode to ensure format consistency across all documents

Tasks 3

Before creating the Document-Term Matrix (DTM), several preprocessing techniques were applied to the corpus to reduce the noise in the data. The preprocessing steps are summarized in *Table 3.1*.

Table 3.1 – Text Preprocessing Process Steps

Step	Description of Steps	Purpose Explanation
1	Lowercasing	Converts all text to lowercase to prevent duplicate representations of the same word

2	Remove URLs	Removes web addresses as they do not contribute meaningful linguistic information and would introduce noise into the DTM
3	Removal of Numbers	Numbers do not contribute to linguistic patterns across the selected genres
2	Removal of Punctuation	Punctuation marks do not contribute to the semantic meaning of the documents
4	Removal of Stopwords & Manually choose the unimportant words	Removes common English stopwords and frequent domain-independent words (e.g., <i>also</i> , <i>can</i> , <i>just</i>) that contribute little to genre differentiation and add noises
5	Removal of Extra Whitespace	Ensures consistent formatting and improves text processing accuracy
6	Stemming	Reduces words to their root forms, allowing similar terms to be treated as a single feature

After processing the documents, a sparse DTM is created to represent the corpus numerically. In a DTM, rows represent documents while column represent terms; Each cell contains the frequency of a term within a document.

The initial DTM contained 20 documents and 3,038 unique terms. After applying sparsity reduction using `removeSparseTerms` function with 0.65 as benchmark which is obtained through trial-and-error, the matrix was reduced to 20 documents and 26 terms. This process removed infrequently occurring words and retained only the most common terms across the corpus.

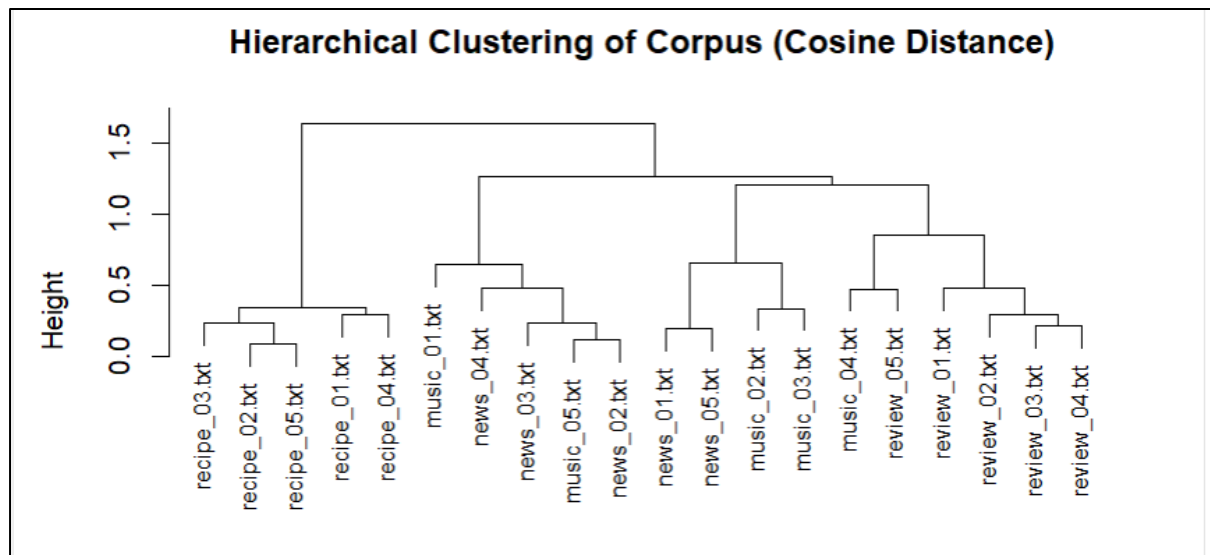
The final retained terms included words: "allow", "becom", "come", "feel", "first", "found", "live", "long", "made", "make", "new", "part", "peopl", "person", "reason", "said", "social", "take", "time", "togeth", "two", "use", "well", "work", "world" and "year". The reduced DTM is attached in the Appendix below.

Tasks 4

Hierarchical clustering was performed on the 20 documents using cosine distance between document vectors derived from the Document-Term Matrix (DTM). Cosine distance was selected over Euclidean distance because it measures the similarity in word usage patterns between documents based on the angle between document vectors rather than their magnitude. This makes it suitable for text data and independent of document length since documents in this corpus vary in length (e.g. news articles are longer than music reviews).

Ward's minimum variance method (`ward.D2`) was used for linkage because it aims to minimise within-cluster variation to produce compact, well-separated clusters.

Figure 4.1 – Hierarchical Clustering of Corpus (Cosine Distance)



To align the clustering solution with the four known genres in *Table 4.1*, the dendrogram was cut at $k = 4$ clusters. The confusion matrix in *Table 4.2* was used to calculate the clustering accuracy shown in *Table 4.3*.

Table 4.1 – Cluster to Genre mapping

Cluster	Color	Assigned Genre
1	Red	News
2	Blue	Music
3	Purple	Review
4	Green	Recipe

Table 4.2 – Confusion Matrix (Actual vs Predicted)

Actual	Predicted			
	Music	News	Recipe	Review
Music	2	2	0	1
News	2	3	0	0
Recipe	0	0	5	0
Review	0	0	0	5

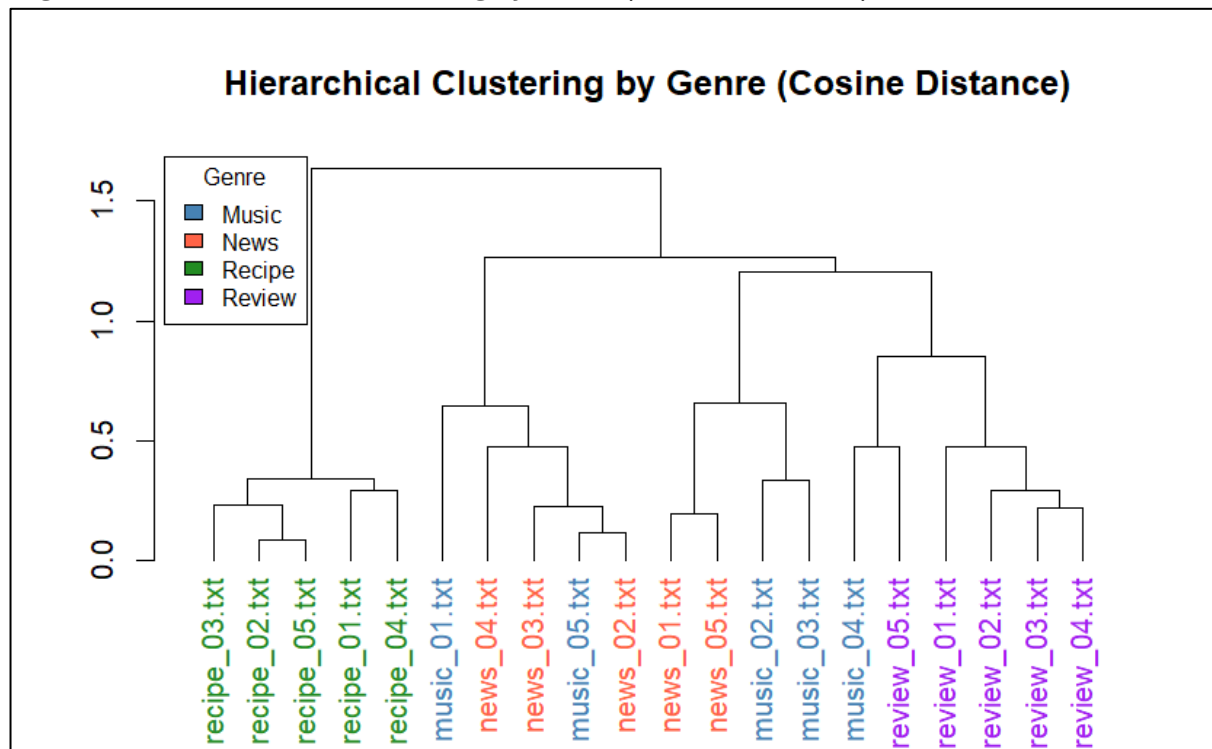
Table 4.3 - Per-Genre Accuracy

Genre	Accuracy
Music	0.4
News	0.6
Recipe	1.0
Review	1.0

Table 4.4 – Top distinctive terms per genre

Genre	Common Words
Music	said, year, world, new, first
News	year, said, social, time, new
Recipe	use, make, togeth, work, time
Review	feel, time people, make, work

Figure 4.2 - Hierarchical Clustering by Genre (Cosine Distance)



The clustering achieved an accuracy of **75%**. Recipe (Cluster 4) and Review (Cluster 3) documents were perfectly separated, achieving 100% accuracy within their clusters. However, Music and News documents were mixed across Clusters 1 and 2, achieved 40% and 60% accuracy, with no clean separation between these two genres, shown in *Table 4.3*.

The dendrogram reveals two major branches: the left branch contains all Recipe documents along with some Music documents, while the right branch contains the remaining Music, News, and Review documents. Recipe form the most cohesive cluster, with all five are grouped together at a low height, indicating high similarity in vocabulary (e.g. cooking-specific terms like *"make"*, *"use"*, *"time"*). Review documents also cluster cleanly, suggesting consistent language patterns across film/product reviews.

The main source of misclassification is the overlap between **Music and News** documents. This is likely because both genres share journalistic language where music documents in this corpus are news-style articles about artists and albums (e.g. BTS tour

announcements), rather than song lyrics or album reviews, meaning they share terms like "said", "first", "new", "year" with news documents. This vocabulary overlap causes the clustering algorithm to group them together rather than by true genre.

Tasks 5

Method & Dictionary

The raw corpus was analysed using the `analyzeSentiment()` function from the `SentimentAnalysis` package in R, which automatically performs text preprocessing and tokenisation. The analysis was performed directly on the corpus, with standard NLP tokenisation handled automatically by the package. The **GI (General Inquirer/Harvard-IV)** dictionary was selected as the primary sentiment measure because it is a general-purpose lexicon suitable for analysing diverse genres. Since the corpus contains Music, News, Recipe, and Review documents, GI was preferred over the finance-specific **HE** and **LM** dictionaries and the conversation-oriented **QDAP** dictionary.

The sentiment score, `SentimentGI`, is calculated as:

$$\text{SentimentGI} = \frac{\text{Positive Words} - \text{Negative Words}}{\text{Total Words}}$$

This measure produces values ranging from -1 to +1, allowing sentiment scores to be compared across documents of different lengths.

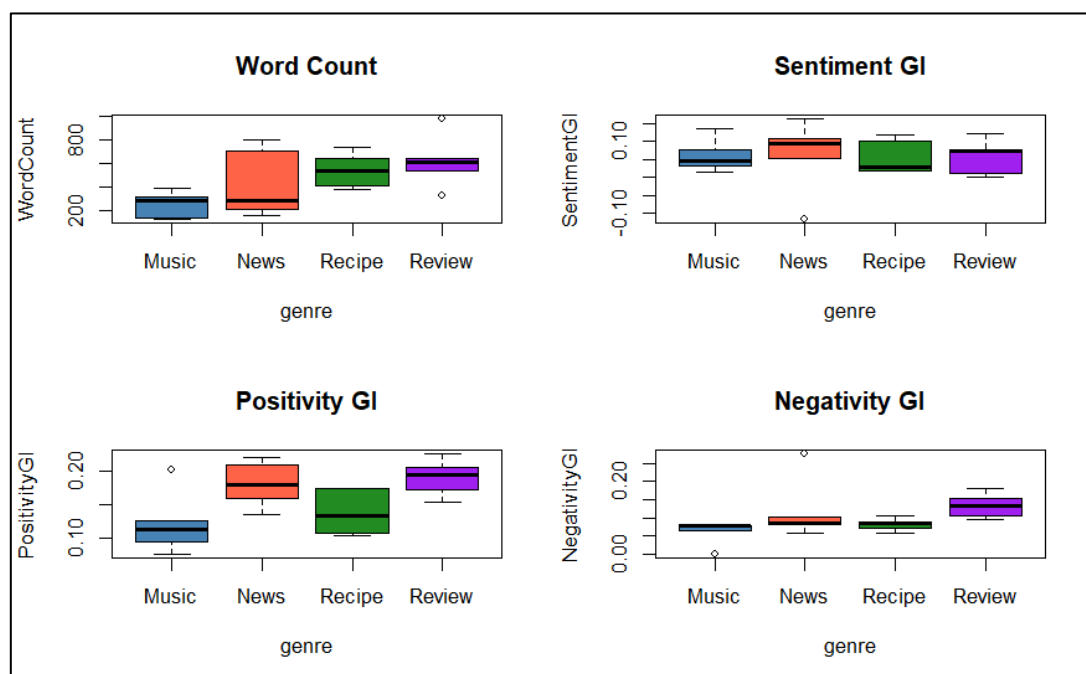


Figure 5.1 – Sentiment Analysis by Genre (GI Dictionary)

The word count boxplot shows document length vary across genres. Music documents are generally shorter, with a median word count of approximately 250 words, whereas

News, Recipe, and Review documents have median lengths ranging from approximately 600 to 750 words. Given these differences, proportional sentiment measures such as **SentimentGI** are more appropriate than raw positive or negative word counts, as they account for document length.

Table 5.1 – Sentiment (GI Dictionary) by Genre

Genre	Mean SentimentGI	SD SentimentGI	Mean PositivityGI	Mean NegativityGI
Music	0.0620	0.0479	0.1218	0.0614
News	0.0608	0.1062	0.1805	0.1194
Recipe	0.0570	0.0493	0.1388	0.0862
Review	0.0564	0.0499	0.1901	0.1336

All four genres exhibited positive mean **SentimentGI** scores, ranging narrowly between 0.056 and 0.062. This suggests that the overall net sentiment is broadly similar across the corpus.

Positivity GI varies more noticeably across genres. Review documents had the highest mean positivity (0.190), followed closely by News (0.180), suggesting these genres use more positive language overall. Music had the lowest positivity (0.122), consistent with its journalistic, factual tone. This pattern is visible in the Positivity GI boxplot, where Music sits clearly lower than News and Review.

Negativity GI also shows meaningful variation. Review documents had the highest negativity (0.134), which is expected as film and product reviews often contain critical language. News showed wide variability in negativity, driven by outliers such as news_03.txt (road bully conviction) which scored very high negativity. Music had the lowest negativity (0.061), consistent with its largely celebratory content about artist releases and tours.

The **Sentiment GI boxplot** shows all genres clustered near zero with overlapping ranges, confirming that net sentiment is similar across genres despite differences in their individual positivity and negativity components.

ANOVA Significance Test

A one-way ANOVA was performed to determine whether mean **SentimentGI** differed significantly across four genres.

- Null Hypothesis (H_0): The mean SentimentGI is equal across all four genres
- Alternative hypothesis (H_1): At least one genre has a different mean SentimentGI

Table 5.2 – One-Way ANOVA Results for SentimentGI by Genre

Source	df	Sum of Squares	Mean Square	F-value	p-value
Genre	3	0.00011	0.000038	0.008	0.999
Residuals	16	0.07391	0.004619		

The ANOVA produced an F-statistic of 0.008 with a p-value of 0.999. Since the p-value is substantially greater than the 0.05 significance level, there is no evidence of a statistically significant difference in mean SentimentGI across genres.

The extremely small F-value indicates that genre explains virtually none of the variation in net sentiment. This finding is consistent with the boxplots, which show highly overlapping SentimentGI distributions. A likely explanation is that positive and negative words occur in similar proportions across all genres, resulting in comparable net sentiment scores. Consequently, analysing PositivityGI and NegativityGI separately provides greater insight into genre-specific differences than relying solely on the overall SentimentGI measure.

Tasks 6

The relationships between documents were analysed using a single-mode network, where each node represents a document and edges are formed when shared tokens exist between documents. To construct the document network, the sparse Document-Term Matrix (DTM) was first converted to a binary matrix (see *Table 6.1*, Appendix), where each cell indicates whether a term appears in a document regardless of frequency. The shared term matrix (see *Table 6.2*, Appendix) was computed by multiplying the binary DTM by its transpose, producing a 20×20 adjacency matrix where each cell represents the number of terms two documents share. The diagonal was set to zero to remove self-connections. A weighted, undirected graph was then built using `graph_from_adjacency_matrix()` from the `igraph` package.

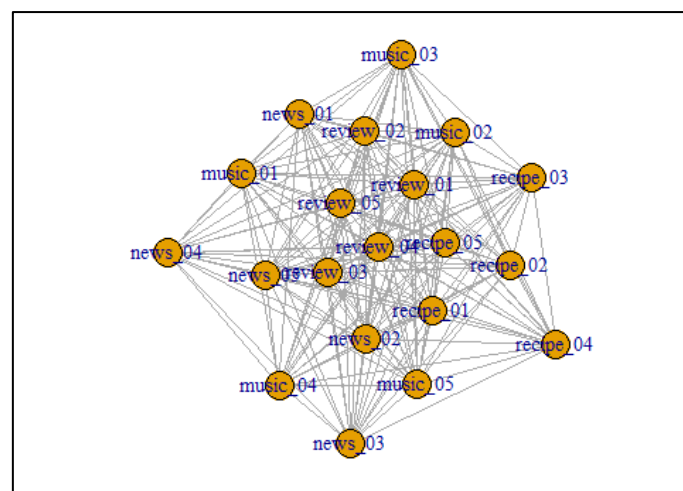


Figure 6.1 – Document Network (Basic)

Figure 6.1 reveals a highly dense network where every document is connected to every other document. This is confirmed by Table 6.3, where 17 out of 20 documents achieve the maximum possible degree of 19, with only recipe_03, recipe_04 and news_04 having lower degrees. This density is expected given the reduced vocabulary of 26 tokens includes many common words (such as “make”, “use) which all documents overlap regardless of genre. As a result, no isolated documents exist, and genre boundaries are completely invisible in the *Figure 6.1*.

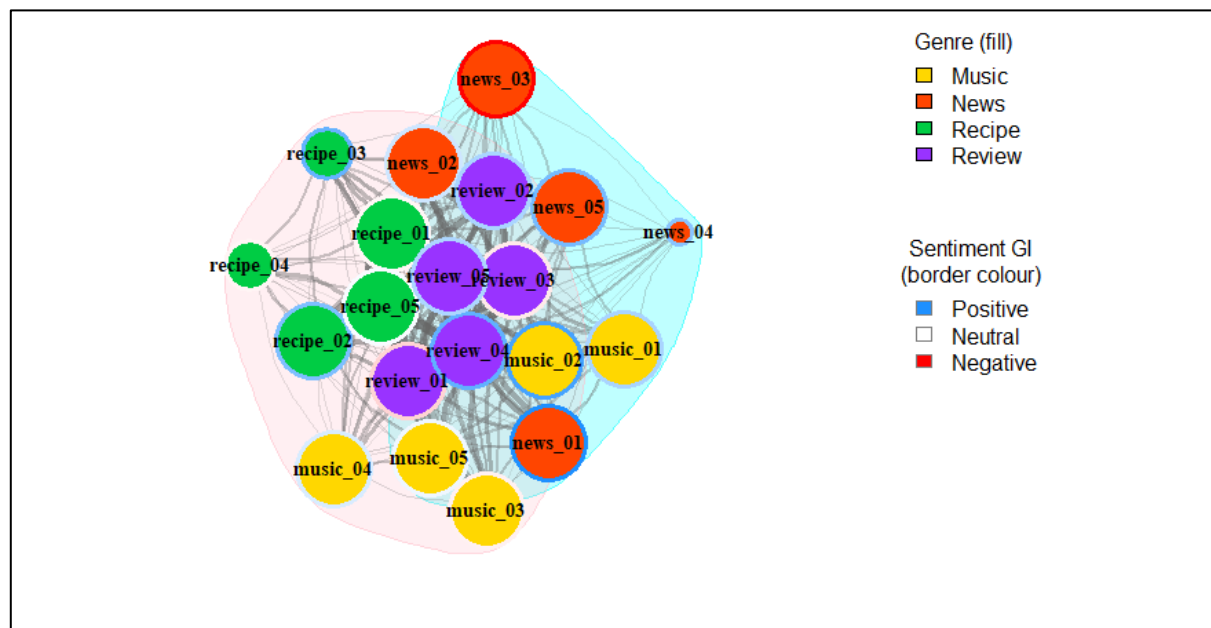


Figure 6.2 – Document Network (Improved)

The improved plot shown in *Figure 6.2* shows four features to highlight structure. First, **node fill colour** represents genre (Music = blue, News = gold, Recipe = green, Review = purple), making genre membership immediately visible. Second, **node size** reflects degree centrality, so more connected documents appear larger. Third, **edge width and transparency** both scale with shared-term weight, with thicker and darker edges showing stronger similarity; the heaviest edge in the network connects review_04 and review_05 with a shared-term count of 17 (*Table 6.2, Appendix*). Fourth, **node border colour** encodes SentimentGI on a red (negative) to blue (positive) scale, adding sentiment information into the network.

Most documents show neutral or positive sentiment. Notably, news_03 stands out with a red border, reflecting its negative sentiment (−0.116), the only negative document. This aligns with its coverage of a road bully conviction, whereas all other documents display neutral or positive sentiment.

Identifiable Groups

The graph primarily reflects writing style and general word frequency rather than topical similarity. Recipe and Review documents (recipe_01, recipe_05, review_04 and

review_05) are among the most heavily connected nodes with the strongest edge weights. These documents share the broadest vocabulary overlap with other documents in the corpus.

Louvain community detection identified two communities. **Community 1 (pink hull)** contains all five News documents together with music_01 and music_05, while **Community 2 (blue hull)** includes all Recipe and Review documents along with music_02, music_03, music_04, and news_02. The perfect grouping of Recipe and Review documents within Community 2 suggests a strong similarity in their descriptive and evaluative language as both genres tend to use instructional and expressive language (e.g. “use”, “feel”). The overall community detection accuracy against true genre labels was **45.0%** (Table 6.5), showing that genre separation remains challenging in such a dense network.

Central Documents

As shown in Table 6.3, **betweenness centrality** is dominated by news_04 (0.361) and recipe_04 (0.171). **Closeness centrality** is also highest for news_04 (0.328) and recipe_04 (0.302). These two documents bridge between the two communities as news_04 in particular lies on the shortest path between many document pairs despite having a slightly lower degree of 17. Their bridging role may be due to their use of a broader vocabulary that overlaps with multiple genres, allowing them to connect the News/Music and Recipe/Review clusters.

On the other hand, review_04, recipe_05, review_05, and review_03 have the **highest eigenvector centrality** cores (1.000, 0.916, 0.915, and 0.838). This shows that these documents are connected to many other important documents, making them part of the core of Community 2. The high values suggest that the Recipe and Review documents share a large amount of common vocabulary. In comparison, news_04 and recipe_04 act as bridges between different communities, while review_04 and recipe_05 are central because they are located within the most densely connected part of the network.

Table 6.3 – Document Centrality Measures (sorted by Degree)

Document	Genre	Degree	Closeness (norm.)	Betweenness (norm.)	Eigenvector	Sentiment GI
music_01	Music	19	0.2639	0.0239	0.4385	0.0763
music_02	Music	19	0.1881	0.0000	0.5878	0.1378
music_03	Music	19	0.2794	0.0302	0.4825	0.0157
music_04	Music	19	0.2969	0.0459	0.4068	0.0469

Document	Genre	Degree	Closeness (norm.)	Betweenness (norm.)	Eigenvector	Sentiment GI
music_05	Music	19	0.2000	0.0000	0.5616	0.0332
news_01	News	19	0.2184	0.0000	0.5233	0.1625
news_02	News	19	0.1827	0.0000	0.6965	0.0534
news_03	News	19	0.2714	0.0136	0.3749	-0.1159
news_05	News	19	0.2135	0.0000	0.5328	0.1076
recipe_01	Recipe	19	0.1545	0.0000	0.8145	0.0185
recipe_02	Recipe	19	0.2676	0.0041	0.6009	0.1026
recipe_05	Recipe	19	0.1759	0.0000	0.9164	0.0271
review_01	Review	19	0.1776	0.0000	0.8136	0.0000
review_02	Review	19	0.1792	0.0000	0.7102	0.0759
review_03	Review	19	0.1532	0.0000	0.8379	0.0121
review_04	Review	19	0.1407	0.0000	1.0000	0.1207
review_05	Review	19	0.1597	0.0000	0.9146	0.0735
recipe_03	Recipe	18	0.2375	0.0012	0.4881	0.1184
recipe_04	Recipe	18	0.3016	0.1714	0.3641	0.0183
news_04	News	17	0.3276	0.3614	0.2857	0.0962

Table 6.4 – Top 5 Documents by Centrality Measure

Rank	Ranked by Betweenness	Ranked by Closeness	Ranked by Eigenvector
1	news_04	news_04	review_04
2	recipe_04	recipe_04	recipe_05
3	music_04	music_04	review_05
4	music_03	music_03	review_03
5	music_01	news_03	recipe_01

Table 6.5 – Community Membership with Sentiment

Document	Genre	Community	Degree	Sentiment GI
music_01	Music	1	19	0.0763

Document	Genre	Community	Degree	Sentiment GI
music_05	Music	1	19	0.0332
news_01	News	1	19	0.1625
news_03	News	1	19	-0.1159
news_04	News	1	17	0.0962
news_05	News	1	19	0.1076
music_02	Music	2	19	0.1378
music_03	Music	2	19	0.0157
music_04	Music	2	19	0.0469
news_02	News	2	19	0.0534
recipe_01	Recipe	2	19	0.0185
recipe_02	Recipe	2	19	0.1026
recipe_03	Recipe	2	18	0.1184
recipe_04	Recipe	2	18	0.0183
recipe_05	Recipe	2	19	0.0271
review_01	Review	2	19	0.0000
review_02	Review	2	19	0.0759
review_03	Review	2	19	0.0121
review_04	Review	2	19	0.1207
review_05	Review	2	19	0.0735

Table 6.6 – Community Detection vs Genre (Confusion Matrix)

	Detected Community	
	1	2
Music	2	3
News	4	1
Recipe	0	5
Review	0	5

Tasks 7

The token network was analysed as a single-mode network where nodes represent the 26 retained tokens and edges represent token co-occurrence within documents. The binary DTM from Task 6 was transposed into a Term-Document Matrix and multiplied by its transpose to produce a 26×26 co-occurrence matrix (see Table 7.1, Appendix). Each cell records the number of documents in which a pair of tokens appears together. Self-connections were removed by setting the diagonal to zero, and a weighted, undirected graph was created using `graph_from_adjacency_matrix()` from the *igraph* package.

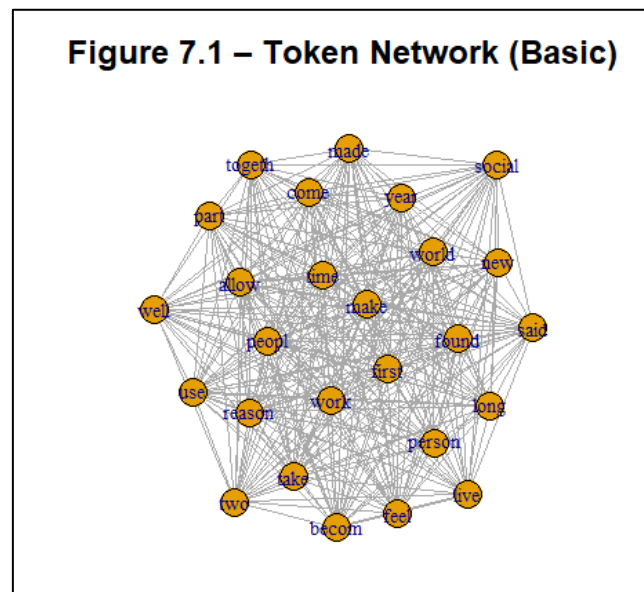


Figure 7.1 – Token Network (Basic)

Figure 7.1 shows a highly dense network in which every token is connected to every other token. This is reflected in Table 7.2, where all tokens have the maximum degree of 25. No isolated tokens exist. This saturation is a direct consequence of the sparsity threshold used in Task 3 which retained only tokens appearing in at least 35% of documents. As a result, most tokens co-occur with one another at least once, making thematic groupings difficult to observe in the basic network.

Although all tokens are connected, edge weights vary substantially. The strongest co-occurrences involve **"make"**, **"first"**, **"time"**, and **"year"**, indicating that these words frequently appear together across multiple documents. In contrast, pairs such as **"well"–"said"**, **"social"–"together"**, and **"well"–"social"** occur together only once, suggesting weaker relationships and more genre-specific usage.

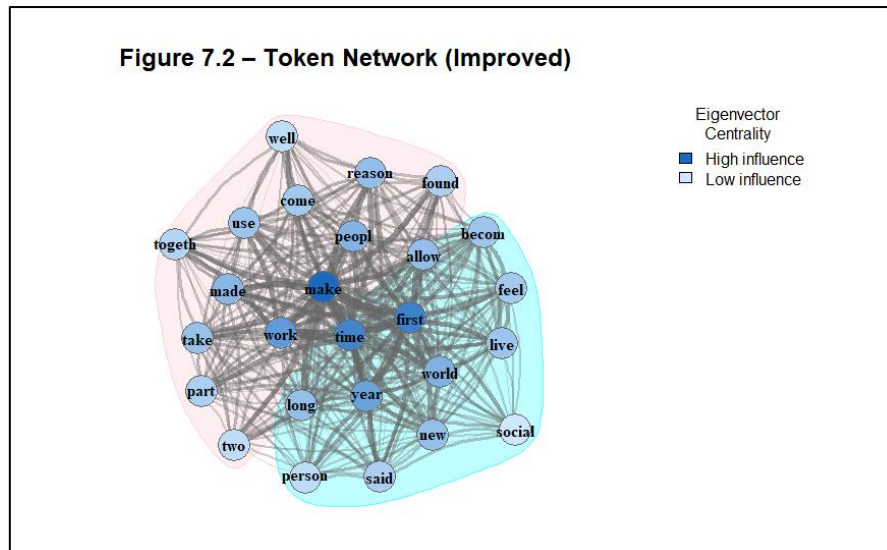


Figure 7.2 – Token Network (Improved)

The improved plot encodes four visual features to reveal structure within the dense network. First, **node size** scales with degree centrality. Second, **node colour** encodes eigenvector centrality on a light-to-deep blue gradient, where darker nodes have higher influence within the network. Third, **edge width and transparency** scale with co-occurrence weight, so thicker and darker edges indicate token pairs appearing together in more documents. Fourth, **community hulls** show the two groups identified by Louvain community detection.

Identifiable Group

Louvain community detection identified two communities. **Community 1** (pink hull) contains 14 tokens: "allow", "come", "found", "made", "make", "part", "peopl", "reason", "take", "togeth", "two", "use", "well", and "work". **Community 2** (blue hull) contains 12 tokens: "becom", "feel", "first", "live", "long", "new", "person", "said", "social", "time", "world", and "year".

The two groups reflect different writing styles within the corpus. Community 1 is dominated by procedural and descriptive language commonly found in Recipe and Review documents, whereas Community 2 contains more journalistic and narrative vocabulary associated with Music and News documents. This pattern closely mirrors the document communities identified in Task 6, suggesting that differences in vocabulary usage drive the observed document clustering.

Central Tokens

Since every token has the same degree of 25, degree centrality does not distinguish token importance. Betweenness, closeness, and eigenvector centrality therefore provide more meaningful insights. (Tables 7.2-7.3)

Betweenness and closeness centrality identify "social", "well", and "togeth" as the main bridge tokens in the network (Table 7.3). "Social" has the highest betweenness (0.122) and closeness (0.287), indicating that it helps connect different groups of tokens. As it is mainly associated with News documents, it links the two vocabulary communities. Its low eigenvector score (0.496, the lowest in the network) confirms it is peripheral rather than central in terms of neighbourhood influence. Similarly, "well" and "togeth" also act as bridge tokens, helping connect the two communities through their use across multiple genres.

Eigenvector centrality highlights the most influential tokens in the network. "Make" has the highest eigenvector centrality (1.000), followed by "first" (0.901), "time" (0.890), "work" (0.799), and "year" (0.774). These tokens broadly distributed but are with other highly connected tokens and form the core vocabulary of the corpus. For example, "make" has some of the highest co-occurrence counts in the network (e.g., make–first = 11, make–time = 11, make–year = 10, make–work = 10).

Peripheral tokens, those with the lowest eigenvector scores, such as "social" (0.496), "two" (0.537), "person" (0.542), and "well" (0.540) suggesting more genre-specific usage. " For example, "social" appears mainly in News documents discussing social media and policy issues, resulting in a more peripheral position despite its strong bridging role.

Overall, the token network reveals a structure consistent with the document network in Task 6. Tokens such as "make", "use", and "work" contribute to the similarity among Recipe and Review documents, while "said", "year", "new", and "social" are more characteristic of Music and News documents. This suggests that the document communities identified previously are largely driven by differences in vocabulary across genres.

Table 7.2 – Token Centrality Measures (sorted by Degree)

Token	Degree	Closeness (norm.)	Betweenness (norm.)	Eigenvector	Community
allow	25	0.1938	0.0000	0.6579	1
becom	25	0.2016	0.0000	0.6361	2
come	25	0.2101	0.0000	0.6181	1
feel	25	0.2101	0.0008	0.6160	2
first	25	0.1420	0.0000	0.9005	2
found	25	0.2212	0.0064	0.5979	1

Token	Degree	Closeness (norm.)	Betweenness (norm.)	Eigenvector	Community
live	25	0.2016	0.0008	0.6346	2
long	25	0.1969	0.0000	0.6558	2
made	25	0.1908	0.0015	0.6872	1
make	25	0.1323	0.0000	1.0000	1
new	25	0.2000	0.0000	0.6567	2
part	25	0.2193	0.0076	0.5948	1
peopl	25	0.1866	0.0000	0.7038	1
person	25	0.2475	0.0199	0.5416	2
reason	25	0.2101	0.0000	0.6582	1
said	25	0.2451	0.0056	0.5977	2
social	25	0.2874	0.1219	0.4957	2
take	25	0.2033	0.0008	0.6432	1
time	25	0.1437	0.0000	0.8904	2
togeth	25	0.2604	0.0336	0.5722	1
two	25	0.2500	0.0051	0.5374	1
use	25	0.2033	0.0000	0.6346	1
well	25	0.2778	0.0733	0.5400	1
work	25	0.1613	0.0000	0.7992	1
world	25	0.1799	0.0000	0.7169	2
year	25	0.1773	0.0000	0.7737	2

Table 7.3 – Top 5 Tokens by Centrality Measure

Rank	Ranked by Betweenness	Ranked by Closeness	Ranked by Eigenvector
1	social	social	make
2	well	well	first
3	togeth	togeth	time

Rank	Ranked by Betweenness	Ranked by Closeness	Ranked by Eigenvector
4	person	two	work
5	part	person	year

Table 7.4 – Token Community Membership

Token	Community	Degree	Eigenvector
allow	1	25	0.6579
come	1	25	0.6181
found	1	25	0.5979
made	1	25	0.6872
make	1	25	1.0000
part	1	25	0.5948
peopl	1	25	0.7038
reason	1	25	0.6582
take	1	25	0.6432
togeth	1	25	0.5722
two	1	25	0.5374
use	1	25	0.6346
well	1	25	0.5400
work	1	25	0.7992
becom	2	25	0.6361
feel	2	25	0.6160
first	2	25	0.9005
live	2	25	0.6346
long	2	25	0.6558
new	2	25	0.6567
person	2	25	0.5416
said	2	25	0.5977

Token	Community	Degree	Eigenvector
social	2	25	0.4957
time	2	25	0.8904
world	2	25	0.7169
year	2	25	0.7737

Tasks 8

A bipartite (two-mode) network was constructed to represent relationship between documents and tokens: 20 document nodes and 26 token nodes, resulting 46 nodes in total. Edges connect a document to a token only when that token appears in the document. The DTM was converted into a long-format edge list where each row represents one document-token pair, producing 254 edges in total.

Unlike Tasks 6 and 7, no document-document or token-token edges exist as every edge strictly crosses between the two node types.

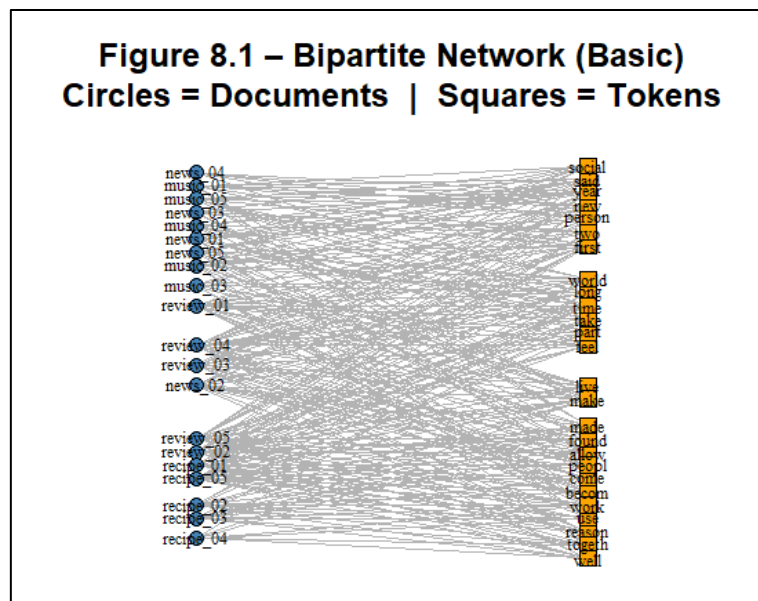


Figure 8.1 – Bipartite Network (Basic)

Figure 8.1 uses the standard bipartite layout with documents on the left and tokens on the right. While it clearly separates the two node types, the dense crossing edges make individual relationships difficult to interpret. However, the graph shows that most documents are linked to multiple tokens and most tokens appear in multiple documents, highlighting the strong overlap within the corpus.

Figure 8.2 – Bipartite Network (Improved)

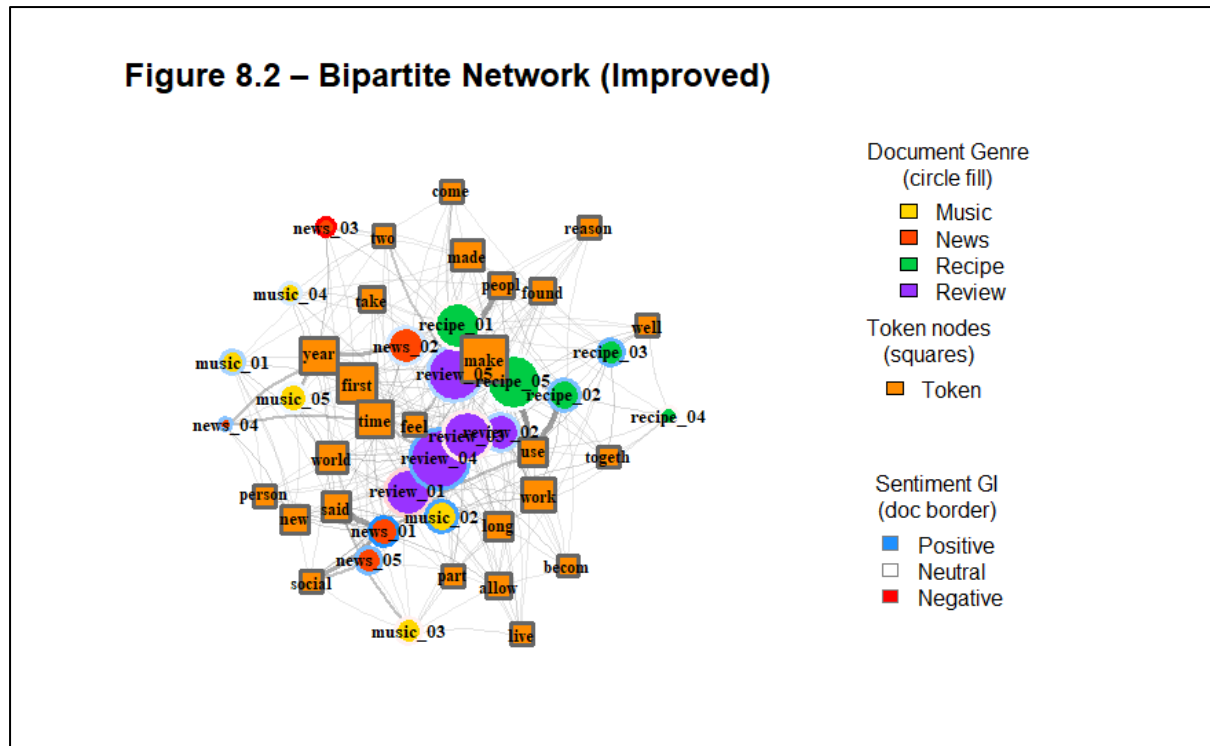


Figure 8.2 – Bipartite Network (Improved)

The improved plot (Figure 8.2) uses a Fruchterman-Reingold layout to better reveal the network structure. First, **document node colour encodes genre** (Music = yellow, News = red, Recipe = green, Review = purple). Second, **node size scales with degree centrality** for both node types, larger document nodes use more tokens and larger token nodes appear in more documents. Third, **document node border colour encodes SentimentGI on a red-to-blue scale**, where blue borders indicate positive sentiment and red indicates negative sentiment. Fourth, **edge width and transparency scale with term frequency from the original DTM**, so thicker darker edges indicate tokens indicating more frequent usage.

The graph reveals a dense central cluster dominated by Recipe (green) and Review (purple) documents, which are connected through interconnected through shared tokens such as "make", "use", "work", and "time". Music (yellow) and News (red) document nodes appear more peripherally, with smaller sizes reflecting their lower degree. news_04 and music_03 are the most peripheral documents in the network, consistent with their low degree in Table 8.1.

Identifiable Groups

The bipartite projection in Table 8.3 shows strong relationship between Review and Recipe documents. The strongest document pair is review_04 and review_05, which shares 17 tokens, followed by recipe_05–review_04 and recipe_05–review_05 with 16 each. All ten of the top document pairs involve only Recipe and Review documents, with no Music or News documents appearing at all, indicating overlap in the vocabulary of

instructional and evaluative language. This pattern is also evident in Figure 8.2, where recipe_01, recipe_05, review_01, review_03, review_04, and review_05 form a dense central cluster connected by numerous high-frequency shared tokens. In contrast, Music and News documents are positioned closer to the network periphery and are linked to the core mainly through more general cross-genre tokens such as "first", "year", and "world".

Central Tokens

Table 8.4 shows that *make* is the most central token, appearing in 16 of the 20 documents, followed by *first* (14), *time* and *year* (13 each), and *work* (12). Their high degree places them at the centre of the bipartite network, consistent with the eigenvector centrality results from Task 7.

By betweenness centrality, *made* (0.057), *person* (0.056), and *reason* (0.047) are the most important bridging tokens. Although they appear in fewer documents, they connect otherwise weakly linked parts of the network. In contrast, tokens such as *social*, *together*, *feel*, *well*, and *two* have lower degree and are more genre-specific, occupying peripheral positions.

Central Document

Table 8.1 identifies review_04 as the most connected document (degree = 22), followed by recipe_05 and review_05 (degree = 20 each). These documents contain the broadest vocabulary and therefore have the highest connectivity.

In terms of betweenness centrality, review_03 (0.146), recipe_01 (0.079), and review_04 (0.079) act as key bridge documents connecting different token groups. Conversely, news_04 has the lowest degree (5) and zero betweenness, making it the most peripheral document in the network. From a sentiment perspective, news_03 is the only document with a negative SentimentGI score (-0.116), while the remaining documents are neutral or positive.

Table 8.1 – Document Node Centrality in Bipartite Network

Document	Genre	Degree	Closeness (norm.)	Betweenness (norm.)	Sentiment GI
review_04	Review	22	0.4891	0.0785	0.1207
recipe_05	Recipe	20	0.4688	0.0737	0.0271
review_05	Review	20	0.4128	0.0284	0.0735
review_03	Review	18	0.5114	0.1463	0.0121
review_01	Review	17	0.4592	0.0358	0.0000

Document	Genre	Degree	Closeness (norm.)	Betweenness (norm.)	Sentiment GI
recipe_01	Recipe	17	0.4787	0.0790	0.0185
news_02	News	14	0.4455	0.0430	0.0534
review_02	Review	14	0.4245	0.0176	0.0759
recipe_02	Recipe	12	0.3846	0.0087	0.1026
music_02	Music	12	0.4206	0.0199	0.1378
news_01	News	11	0.3125	0.0020	0.1625
music_05	Music	11	0.4245	0.0293	0.0332
news_05	News	10	0.4054	0.0167	0.1076
recipe_03	Recipe	10	0.4206	0.0316	0.1184
music_03	Music	10	0.4167	0.0322	0.0157
music_01	Music	9	0.4054	0.0186	0.0763
music_04	Music	8	0.3750	0.0113	0.0469
news_03	News	7	0.3982	0.0140	-0.1159
recipe_04	Recipe	7	0.3659	0.0084	0.0183
news_04	News	5	0.3214	0.0000	0.0962

Table 8.2 – Token Node Centrality in Bipartite Network

Token	Degree (docs containing token)	Closeness (norm.)	Betweenness (norm.)
make	16	0.4245	0.0102
first	14	0.4545	0.0346
time	13	0.4128	0.0132
year	13	0.3982	0.0269
work	12	0.3750	0.0083
made	11	0.4639	0.0567
world	11	0.4245	0.0209
long	10	0.4206	0.0209

Token	Degree (docs containing token)	Closeness (norm.)	Betweenness (norm.)
new	10	0.4545	0.0179
said	10	0.3629	0.0049
use	10	0.4018	0.0080
allow	9	0.4369	0.0269
found	9	0.4245	0.0235
peopl	9	0.4545	0.0274
take	9	0.4455	0.0206
becom	8	0.4327	0.0186
come	8	0.4592	0.0250
feel	8	0.3750	0.0070
live	8	0.4167	0.0088
part	8	0.4245	0.0121
person	8	0.4412	0.0564
reason	8	0.4737	0.0467
social	8	0.4286	0.0225
togeth	8	0.3947	0.0092
two	8	0.4286	0.0213
well	8	0.4286	0.0103

Table 8.3 – Top 10 Document Pairs by Shared Tokens (Bipartite Projection)

Document A	Document B	Shared Tokens
review_04	review_05	17
recipe_05	review_04	16
recipe_05	review_05	16
review_01	review_04	16
review_04	review_03	15
recipe_05	review_01	14

Document A	Document B	Shared Tokens
recipe_05	review_03	14
review_04	recipe_01	14
review_05	recipe_01	14
recipe_05	recipe_01	13

Table 8.4 – Tokens by Number of Documents (Degree in Bipartite Network)

Token	Number of Documents
make	16
first	14
time	13
year	13
work	12
made	11
world	11
long	10
new	10
said	10
use	10
allow	9
found	9
peopl	9
take	9
becom	8
come	8
feel	8
live	8
part	8

Token	Number of Documents
person	8
reason	8
social	8
togeth	8
two	8
well	8

Final Analysis Summary

Important Documents

Across Tasks 6–8, review_04 is the most central document. It achieved the highest eigenvector centrality in the document network, the highest degree in the bipartite network (22), and formed the strongest document pair with review_05 (17 shared tokens). recipe_05 and review_05 also ranked consistently among the most connected documents.

In contrast, news_04 and recipe_04 served as important bridge documents. Although news_04 had relatively low connectivity, it recorded the highest betweenness centrality, indicating its role in linking the two major document groups. news_03 was the only document with a negative SentimentGI score (-0.116), driven by its coverage of a road bully conviction, reflecting the negative nature of its content.

Important Tokens

The token *make* was the most influential term across all analyses, achieving the highest eigenvector centrality and appearing in 16 of the 20 documents. Other highly central tokens included *first*, *time*, *work*, and *year*.

Bridging tokens such as *social*, *well*, and *togeth* connected the two vocabulary communities despite their lower frequencies. Conversely, tokens such as *two*, *person*, and *social* occupied more peripheral positions due to their more specialised usage.

Groups and Clusters

Two clear document groups emerged throughout the analyses. Recipe and Review documents formed a strong cluster driven by shared instructional and evaluative vocabulary. This grouping was consistently supported by hierarchical clustering, community detection, and the bipartite network.

Music and News documents formed a second cluster based on journalistic vocabulary such as *said*, *year*, *new*, and *world*. However, the distinction between these genres was

weaker because the Music documents were largely news-style articles, resulting in substantial vocabulary overlap.

Effectiveness of Clustering vs Network Analysis

Both methods identified the same overall structure: a Recipe/Review group and a Music/News group.

Hierarchical clustering achieved higher classification accuracy (75%) and provided clear group assignments through the dendrogram. However, it could not reveal the relative importance of documents or the connections between groups.

Network analysis provided insights beyond clustering by revealing connectivity and node importance. The document network showed that all documents share some vocabulary, while centrality measures identified review_04 and recipe_05 as core documents and news_04 and recipe_04 as key bridges between groups. The token and bipartite networks further explained the vocabulary patterns driving document similarity.

Overall, hierarchical clustering was more effective for genre classification, whereas network analysis was more effective for understanding relationships, influence, and connectivity within the corpus.

Improvements to Text Processing

The current bag-of-words approach ignores word order and context. Several NLP techniques could improve genre discrimination:

- **TF-IDF (Term Frequency-Inverse Document Frequency)** would reduce the influence of common terms such as "make" and "time" while increasing the importance of genre-specific words, likely improving clustering performance. TF-IDF down-weights terms that appear frequently across many documents and up-weights terms distinctive to specific documents, this would help distinguish documents more effectively instead of raw term frequencies. (Sparck Jones, 1972).
- **Word embeddings (e.g., Word2Vec or GloVe)** would capture semantic similarity between words, allowing documents to be compared based on meaning rather than exact vocabulary. Words with similar meanings are treated as nearby vectors in a continuous semantic space, in other words, related words even when using different vocabulary, so documents be compared based on meaning. (Mikolov et al., 2013; Pennington et al., 2014).
- **Transformer models** such as BERT would generate context-sensitive representations, distinguishing words according to their surrounding context and potentially providing the highest classification accuracy, although at greater

computational cost. Unlike bag-of-words, BERT produces a different vector for the same word depending on how it is used in a sentence, directly addressing the vocabulary overlap between Music and News documents in this corpus (Devlin et al., 2019).

These methods would provide richer linguistic information than the current DTM-based approach and could improve both clustering and network analyses (Zong et al., 2021).

References

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1, 4171–4186.

<https://doi.org/10.18653/v1/N19-1423>

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space*. arXiv. <https://arxiv.org/abs/1301.3781>

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532–1543. <https://doi.org/10.3115/v1/D14-1162>

Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21. <https://doi.org/10.1108/eb026526>

Zong, C., Xia, R., & Zhang, J. (2021). *Text data mining*. Springer Nature.

<https://doi.org/10.1007/978-981-16-0100-2>

Genre: Music

allkpop. (2026, February 13). BTS reunites for GQ March 2026 cover, opens up about comeback, pressure, and new album. allkpop.

<https://www.allkpop.com/article/2026/02/bts-reunites-for-gq-march-2026-cover-opens-up-about-comeback-pressure-and-new-album>

Malay Mail. (2026, January 5). BTS marks full-group return with album on March 20, world tour info on Jan 14. Malay Mail.

<https://www.malaymail.com/news/showbiz/2026/01/05/bts-marks-full-group-return-with-album-on-march-20-world-tour-info-on-jan-14/204329>

The Star. (2026, June 4). ARMY's frantic bid for BTS tickets. The Star.

<https://www.thestar.com.my/news/nation/2026/06/04/armys-frantic-bid-for-bts-tickets>

triple j. (n.d.). Behind Audrey Hobert's cover of Malcolm Todd's 'Earrings' [Interview].

ABC triple j. <https://www.abc.net.au/triplej/like-a-version/behind-audrey-hobert-cover-malcolm-todd-earrings-interview/106564734>

BBC News. (n.d.). [Music article]. BBC News.
<https://www.bbc.com/news/articles/c9v04gney9yo>

Genre: Recipe

Matsumoto, M. (2026, February 1). Tonkotsu ramen. NoRecipes.
<https://norecipes.com/tonkotsu-ramen-recipe/>

Matsumoto, M. (2026, April 3). Tamagoyaki (Japanese omelette). NoRecipes.
<https://norecipes.com/tamagoyaki-recipe/>

Segal, J. (2024, October 23). Kung Pao chicken. Once Upon a Chef.
<https://www.onceuponachef.com/recipes/kung-pao-chicken.html>

True, M. (2004, April). Shanghai soup dumplings (Xiao Long Bao). Saveur.
<https://www.saveur.com/article/Recipes/Shanghai-Soup-Dumplings/>

Serious Eats. (n.d.). Tamago kake gohan (TKG) egg rice breakfast recipe. Serious Eats.
<https://www.serious-eats.com/tamago-kake-gohan-egg-rice-tkg-recipe-breakfast>

Genre: News

Yusry, M. (2026, June 1). Malaysia's under-16 social media rule starts today: What parents need to know. Malay Mail.
<https://www.malaymail.com/news/malaysia/2026/06/01/malaysias-under-16-social-media-rule-starts-today-what-parents-need-to-know/221674>

The Diplomat. (2026, February). Japan's inbound tourism is booming, but Japanese are less interested in going abroad. The Diplomat.
<https://thediplomat.com/2026/02/japans-inbound-tourism-is-booming-but-japanese-are-less-interested-in-going-abroad/>

Dermawan, A. (2026, June 4). Road bully jailed, fined RM8k and banned from driving for five years. New Straits Times.
<https://www.nst.com.my/news/regional/2026/06/1455270/road-bully-jailed-fined-rm8k-and-banned-driving-five-years>

Lim, I. (2026, June 4). Simplified: New 10-year Malaysian passport at RM350; pay up to RM550 to RM1,350 if you lose it. Malay Mail.
<https://www.malaymail.com/news/malaysia/2026/06/04/simplified-new-10-year-malaysian-passport-at-rm350-pay-up-to-rm550-to-rm1350-if-you-lose-it/222475>

New Straits Times. (2026, June). Meta lashes Australia's bid to make tech giants pay for news. New Straits Times.
<https://www.nst.com.my/world/world/2026/06/1454910/meta-lashes-australias-bid-make-tech-giants-pay-news>

Genre: Movie/ TV Show Review

Zilko, C. (2026, May 13). 'Obsession' review: YouTuber Curry Barker directs one of the best horror films of 2026. IndieWire.

<https://www.indiewire.com/criticism/movies/obsession-review-curry-baker-1235149428/>

Mehta, R. (2026, May 17). 'Colony' review: Yeon Sang-ho's latest zombie saga paints civilizational collapse with too broad a brush. IndieWire.

<https://www.indiewire.com/criticism/movies/colony-review-yeon-sang-ho-1235194485/>

Park, J. (2026, May 11). [Review] 'We Are All Trying Here' answers fear of worthlessness.

The Korea Times. <https://www.koreatimes.co.kr/entertainment/shows-dramas/20260511/we-are-all-trying-here-answers-fear-of-worthlessness>

Menon, R. (2026, April 29). 'The Devil Wears Prada 2' review. Script Magazine.

<https://scriptmag.com/the-devil-wears-prada-2-review>

Feifei. (2026, February 1). A review of "Can This Love Be Translated?" Feifei's Substack.

<https://feifeiwrites.substack.com/p/a-review-of-can-this-love-be-translated>

Appendix

FIT3152_A3

```
library(slam) # for matrices and arrays
library(tm)
```

Warning: package 'tm' was built under R version 4.5.3

Loading required package: NLP

```
library(SnowballC) # for stemming
library(knitr)
library(kableExtra)
```

Warning: package 'kableExtra' was built under R version 4.5.3

```
library(SentimentAnalysis)
```

Warning: package 'SentimentAnalysis' was built under R version 4.5.3

Attaching package: 'SentimentAnalysis'

The following object is masked from 'package:base':

write

Task 3

```
# -Load corpus from folder
corpus_dir <- "Corpus"

docs <- VCorpus(DirSource(corpus_dir, encoding = "UTF-8"),
               readerControl = list(language = "en"))

# Verify documents loaded
cat("Number of documents:", length(docs), "\n")
```

Number of documents: 20

```
cat("Document names:\n")
```

Document names:

```
print(names(docs))
```

```
[1] "music_01.txt" "music_02.txt" "music_03.txt" "music_04.txt"
[5] "music_05.txt" "news_01.txt" "news_02.txt" "news_03.txt"
[9] "news_04.txt" "news_05.txt" "recipe_01.txt" "recipe_02.txt"
[13] "recipe_03.txt" "recipe_04.txt" "recipe_05.txt" "review_01.txt"
[17] "review_02.txt" "review_03.txt" "review_04.txt" "review_05.txt"
```

```
# Text transformations / cleaning

# Convert to lowercase
docs <- tm_map(docs, content_transformer(tolower))

# Remove URLs (common artefact in web-scraped text)
remove_url <- function(x) gsub("http\\S+|www\\.\\S+", "", x)
docs <- tm_map(docs, content_transformer(remove_url))

# Remove numbers
docs <- tm_map(docs, removeNumbers)

# Remove punctuation
docs <- tm_map(docs, removePunctuation)

# Remove common English stopwords
custom_stopwords <- c(stopwords("english"),
                      "also", "can", "just", "like", "even", "still",
                      "want", "know", "will", "one", "need", "get",
                      "now", "fine", "turn")
docs <- tm_map(docs, removeWords, custom_stopwords)

# Strip extra whitespace
docs <- tm_map(docs, stripWhitespace)

# Stem words (reduce to root form)
docs <- tm_map(docs, stemDocument)

# Build the Document-Term Matrix
dtm <- DocumentTermMatrix(docs)

cat("\nDTM dimensions (before sparsity removal):",
    dim(dtm)[1], "documents x", dim(dtm)[2], "terms\n")
```

DTM dimensions (before sparsity removal): 20 documents x 3027 terms

```
# Remove sparse terms
dtm_sparse <- removeSparseTerms(dtm, sparse = 0.65)

cat("DTM dimensions (after sparsity removal):",
    dim(dtm_sparse)[1], "documents x", dim(dtm_sparse)[2], "terms\n")
```

DTM dimensions (after sparsity removal): 20 documents x 26 terms

```
# Check how many tokens remain
cat("Tokens retained:", ncol(dtm_sparse), "\n")
```

Tokens retained: 26

```
cat("Terms:\n")
```

Terms:

```
print(Terms(dtm_sparse))
```

```
[1] "allow"  "becom"  "come"   "feel"   "first"  "found"  "live"   "long"
[9] "made"   "make"   "new"    "part"   "peopl"  "person" "reason" "said"
[17] "social" "take"   "time"   "togeth" "two"    "use"    "well"   "work"
[25] "world"  "year"
```

```
# Inspect and display the DTM
dtm_matrix <- as.matrix(dtm_sparse)
print(dtm_matrix)
```

	Terms											
Docs	allow	becom	come	feel	first	found	live	long	made	make	new	part
music_01.txt	0	0	1	0	3	0	0	0	1	0	2	0
music_02.txt	0	0	0	0	1	1	2	1	0	2	4	0
music_03.txt	0	1	0	1	1	0	2	2	0	0	0	0
music_04.txt	0	0	0	1	0	0	0	1	0	1	0	0
music_05.txt	0	0	0	0	0	0	0	1	1	2	1	1
news_01.txt	5	0	0	0	0	0	0	0	0	0	4	2
news_02.txt	1	0	1	0	2	1	0	1	1	3	0	1
news_03.txt	0	0	0	0	1	1	0	0	1	3	0	0
news_04.txt	0	0	0	0	2	0	0	0	0	0	5	0
news_05.txt	1	0	0	0	1	1	0	0	0	2	0	0
recipe_01.txt	0	0	3	0	2	0	0	0	1	8	0	1
recipe_02.txt	0	1	0	0	0	0	0	0	2	8	0	2
recipe_03.txt	1	0	1	0	0	1	0	0	1	2	0	0
recipe_04.txt	0	0	0	0	0	0	0	2	1	3	0	0
recipe_05.txt	1	1	1	1	3	2	1	1	2	7	1	0
review_01.txt	1	2	0	2	1	0	1	3	1	2	2	1
review_02.txt	2	2	0	4	2	1	1	0	0	2	1	0
review_03.txt	0	1	1	4	1	2	2	3	0	1	1	0
review_04.txt	2	4	1	15	3	0	2	3	2	1	1	2
review_05.txt	1	1	2	7	3	3	1	0	0	5	0	2

	Terms											
Docs	peopl	person	reason	said	social	take	time	togeth	two	use	well	
music_01.txt	0	1	0	1	1	0	0	0	0	0	0	

music_02.txt	0	1	0	5	0	0	0	4	1	0	0
music_03.txt	0	0	0	7	1	0	0	0	0	1	0
music_04.txt	1	0	0	3	0	2	0	0	2	0	0
music_05.txt	0	0	0	4	0	1	1	0	1	0	0
news_01.txt	0	5	0	12	10	2	3	0	0	7	0
news_02.txt	1	0	1	4	0	0	3	0	0	0	1
news_03.txt	0	0	0	0	1	0	0	0	1	0	0
news_04.txt	0	1	0	0	0	0	7	0	0	0	0
news_05.txt	0	0	0	4	7	0	1	0	0	2	0
recipe_01.txt	1	1	1	0	0	1	3	1	1	4	1
recipe_02.txt	3	0	1	0	0	0	1	2	0	12	2
recipe_03.txt	0	0	0	0	0	1	0	4	0	5	1
recipe_04.txt	0	0	0	0	0	0	0	1	0	2	2
recipe_05.txt	4	0	1	0	0	1	5	2	1	13	3
review_01.txt	1	0	0	0	5	2	2	0	1	0	0
review_02.txt	0	0	1	0	0	0	3	0	0	0	3
review_03.txt	1	1	1	0	1	0	1	0	0	1	1
review_04.txt	1	1	1	1	1	1	7	1	0	0	0
review_05.txt	12	2	1	1	0	3	3	3	6	1	0

Terms

Docs	work	world	year
music_01.txt	0	4	2
music_02.txt	2	2	0
music_03.txt	2	1	0
music_04.txt	0	0	2
music_05.txt	0	3	8
news_01.txt	2	0	1
news_02.txt	0	0	9
news_03.txt	0	0	5
news_04.txt	0	0	7
news_05.txt	0	1	2
recipe_01.txt	2	1	2
recipe_02.txt	4	1	0
recipe_03.txt	1	0	0
recipe_04.txt	1	0	0
recipe_05.txt	2	0	0
review_01.txt	2	3	0
review_02.txt	2	1	2
review_03.txt	0	2	1
review_04.txt	2	5	2
review_05.txt	5	0	2

Task 4

```
library(proxy)      # cosine distance
```

Warning: package 'proxy' was built under R version 4.5.3

Attaching package: 'proxy'

The following objects are masked from 'package:stats':

as.dist, dist

The following object is masked from 'package:base':

as.matrix

```
library(igraph)      # network graphs
```

Warning: package 'igraph' was built under R version 4.5.3

Attaching package: 'igraph'

The following objects are masked from 'package:stats':

decompose, spectrum

The following object is masked from 'package:base':

union

```
library(ggplot2)     # plotting
```

Attaching package: 'ggplot2'

The following object is masked from 'package:NLP':

annotate

```
library(RColorBrewer)
library(dendextend)
```

Warning: package 'dendextend' was built under R version 4.5.3

Welcome to dendextend version 1.19.1

Type citation('dendextend') for how to cite the package.

Type browseVignettes(package = 'dendextend') for the package vignette.

The github page is: <https://github.com/talgalili/dendextend/>

Suggestions and bug-reports can be submitted at: <https://github.com/talgalili/dendextend/>

issues

You may ask questions at stackoverflow, use the `r` and `dendextend` tags:

<https://stackoverflow.com/questions/tagged/dendextend>

To suppress this message use: `suppressPackageStartupMessages(library(dendextend))`

Attaching package: 'dendextend'

The following object is masked from 'package:stats':

`cutree`

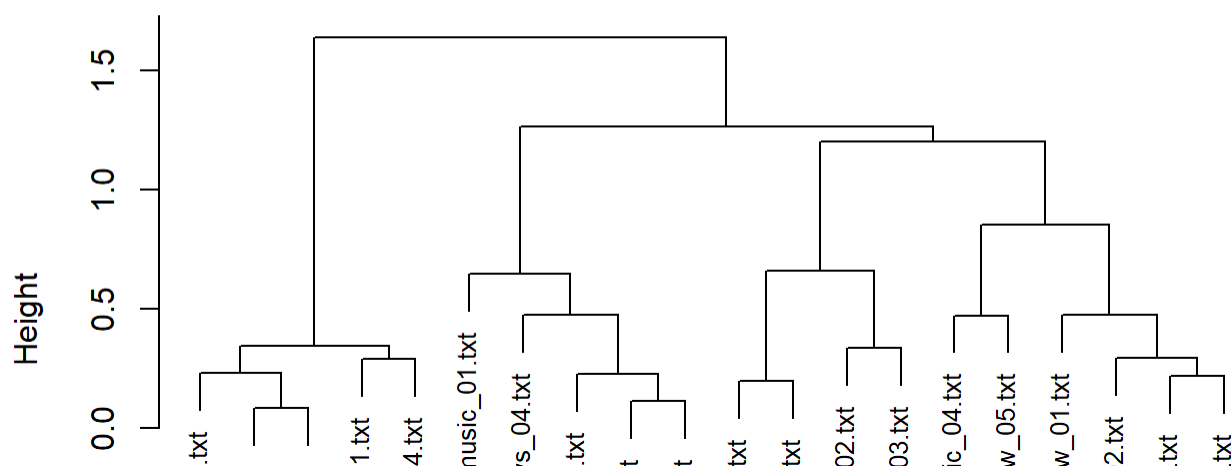
```
# Cosine distance matrix from DTM
dtm_matrix <- as.matrix(dtm_sparse)
cosine_dist <- dist(dtm_matrix, method = "cosine")

# Hierarchical clustering (Ward's method)
hclust_result <- hclust(cosine_dist, method = "ward.D2")

# Genre labels for colouring
genres <- c(rep("Music", 5), rep("News", 5), rep("Recipe", 5), rep("Review", 5))
genre_colours <- c("Music" = "steelblue", "News" = "tomato",
                  "Recipe" = "forestgreen", "Review" = "purple")
label_colours <- genre_colours[genres]

# Plot dendrogram
plot(hclust_result,
     main = "Hierarchical Clustering of Corpus (Cosine Distance)",
     xlab = "", sub = "",
     ylab = "Height",
     cex = 0.8)
```

Hierarchical Clustering of Corpus (Cosine Distance)



```

recipe_03
recipe_02.txt
recipe_05.txt
recipe_0
recipe_0
r
new
news_03
music_05.tx
news_02.tx
news_01.
news_05.
music_
music_
mus
revie
revie
review_c
review_03
review_04

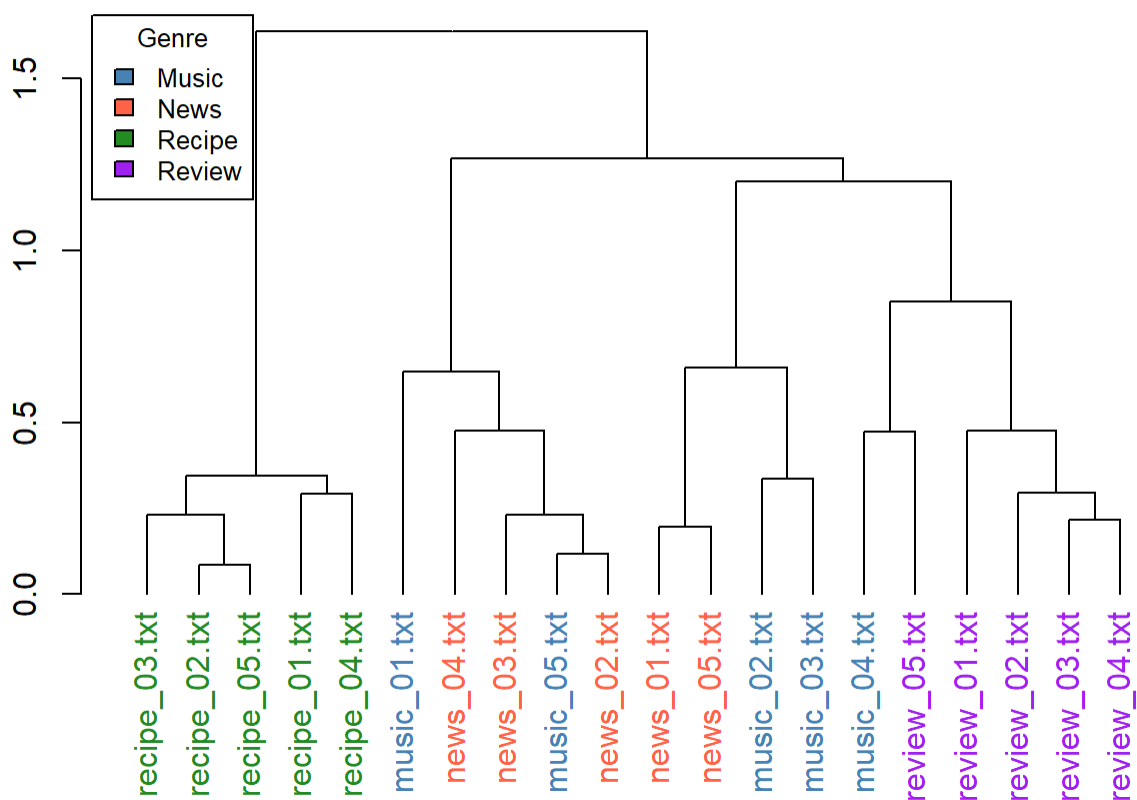
```

```

# Colour leaf labels by genre
dend <- as.dendrogram(hclust_result)
dend <- color_labels(dend, col = label_colours[order.dendrogram(dend)])
plot(dend, main = "Hierarchical Clustering by Genre (Cosine Distance)")
legend("topleft",
      legend = names(genre_colours),
      fill = genre_colours,
      title = "Genre",
      cex = 0.8,
      inset = c(0.01, 0.01), # slight padding from corner
      bg = "white",          # white background so it's readable
      box.lty = 1)

```

Hierarchical Clustering by Genre (Cosine Distance)



```
# Cut tree into k clusters
k <- 4
clusters <- cutree(hclust_result, k = k)
cat("Cluster assignments:\n")
```

Cluster assignments:

```
print(clusters)
```

```
music_01.txt  music_02.txt  music_03.txt  music_04.txt  music_05.txt
           1           2           2           3           1
news_01.txt   news_02.txt   news_03.txt   news_04.txt   news_05.txt
           2           1           1           1           2
recipe_01.txt recipe_02.txt recipe_03.txt recipe_04.txt recipe_05.txt
           4           4           4           4           4
review_01.txt review_02.txt review_03.txt review_04.txt review_05.txt
           3           3           3           3           3
```

```
# Accuracy: compare clusters to true genres
genre_labels <- rep(c("Music", "News", "Recipe", "Review"), each = 5)
cluster_df <- data.frame(doc = names(clusters),
                        cluster = clusters,
                        genre = genre_labels)
cat("\nCluster vs Genre table:\n")
```

Cluster vs Genre table:

```
print(table(cluster_df$cluster, cluster_df$genre))
```

	Music	News	Recipe	Review
1	2	3	0	0
2	2	2	0	0
3	1	0	0	5
4	0	0	5	0

```
# Accuracy: majority genre per cluster
cluster_accuracy <- function(clusters, genres) {
  df <- data.frame(cluster = clusters, genre = genres)
  correct <- sum(sapply(unique(clusters), function(c) {
    sub <- df[df$cluster == c, "genre"]
    max(table(sub))
  })))
  correct / length(genres)
}
```

```
acc <- cluster_accuracy(clusters, genre_labels)
cat(sprintf("\nClustering accuracy: %.1f%%\n", acc * 100))
```

Clustering accuracy: 75.0%

```
# Confusion Matrix

# Map cluster numbers to genre names by majority vote
cluster_to_genre <- sapply(1:k, function(c) {
  sub <- genre_labels[clusters == c]
  names(which.max(table(sub)))
})
cat("Cluster to Genre mapping:\n")
```

Cluster to Genre mapping:

```
print(data.frame(Cluster = 1:k, Assigned_Genre = cluster_to_genre))
```

	Cluster	Assigned_Genre
1	1	News
2	2	Music
3	3	Review
4	4	Recipe

```
# Predict genre for each document
predicted_genre <- factor(cluster_to_genre[clusters],
                          levels = c("Music", "News", "Recipe", "Review"))
actual_genre <- factor(genre_labels,
                      levels = c("Music", "News", "Recipe", "Review"))

# Confusion matrix (Actual = rows, Predicted = columns)
conf_table <- table(Actual = actual_genre, Predicted = predicted_genre)
cat("\nConfusion Matrix (Actual vs Predicted):\n")
```

Confusion Matrix (Actual vs Predicted):

```
print(conf_table)
```

	Predicted			
Actual	Music	News	Recipe	Review
Music	2	2	0	1
News	2	3	0	0
Recipe	0	0	5	0
Review	0	0	0	5

```
# Per-cluster accuracy
per_cluster_acc <- sapply(c("Music","News","Recipe","Review"), function(g) {
  row <- conf_table[g, ]
  round(max(row) / sum(row), 2)
})
cat("\nPer-Genre Accuracy:\n")
```

Per-Genre Accuracy:

```
print(data.frame(Genre = names(per_cluster_acc), Accuracy = per_cluster_acc))
```

	Genre	Accuracy
Music	Music	0.4
News	News	0.6
Recipe	Recipe	1.0
Review	Review	1.0

```
# Overall accuracy
overall_acc <- sum(diag(conf_table)) / sum(conf_table)
cat(sprintf("\nOverall Accuracy: %.1f%%\n", overall_acc * 100))
```

Overall Accuracy: 75.0%

```
# Table 4.4: Top distinctive terms per genre
genre_labels <- rep(c("Music","News","Recipe","Review"), each = 5)

# Sum term frequencies by genre
genre_term_matrix <- aggregate(dtm_matrix,
                               by = list(Genre = genre_labels),
                               FUN = sum)

# Set genre names as row names
rownames(genre_term_matrix) <- genre_term_matrix$Genre
genre_term_matrix$Genre <- NULL

# Print top 5 terms per genre
for (g in c("Music", "News", "Recipe", "Review")) {
  row <- as.numeric(genre_term_matrix[g, ])      # convert to numeric vector
  names(row) <- colnames(genre_term_matrix)      # restore term names
  row_sorted <- sort(row, decreasing = TRUE)     # now sort works
  cat(g, ":", paste(names(row_sorted)[1:5], collapse = ", "), "\n")
}
```

```
Music : said, year, world, new, first
News  : year, said, social, time, new
Recipe: use, make, togeth, work, time
- . - . . - . . - . .
```

Review : feel, time, peopl, make, work

Task 5

```
# Analyse sentiment directly from corpus
# Standard NLP tokenizing performed automatically
SentimentA <- analyzeSentiment(docs)

cat("Sentiment scores:\n")
```

Sentiment scores:

```
print(SentimentA)
```

	WordCount	SentimentGI	NegativityGI	PositivityGI	SentimentHE	NegativityHE
1	131	0.07633588	0.00000000	0.07633588	0.00000000	0.00000000
2	283	0.13780919	0.06360424	0.20141343	0.003533569	0.010600707
3	319	0.01567398	0.07836991	0.09404389	0.003134796	0.003134796
4	128	0.04687500	0.07812500	0.12500000	0.015625000	0.000000000
5	392	0.03316327	0.07908163	0.11224490	0.007653061	0.000000000
6	800	0.16250000	0.05750000	0.22000000	0.000000000	0.012500000
7	711	0.05344585	0.08157525	0.13502110	-0.001406470	0.033755274
8	207	-0.11594203	0.27536232	0.15942029	-0.004830918	0.004830918
9	156	0.09615385	0.08333333	0.17948718	-0.006410256	0.006410256
10	288	0.10763889	0.10069444	0.20833333	0.006944444	0.003472222
11	648	0.01851852	0.08950617	0.10802469	0.006172840	0.003086420
12	536	0.10261194	0.07089552	0.17350746	0.005597015	0.007462687
13	414	0.11835749	0.05555556	0.17391304	0.004830918	0.002415459
14	383	0.01827676	0.08616188	0.10443864	0.007832898	0.000000000
15	739	0.02706360	0.10690122	0.13396482	0.013531800	0.001353180
16	541	0.00000000	0.15341959	0.15341959	0.007393715	0.005545287
17	646	0.07585139	0.09597523	0.17182663	-0.001547988	0.007739938
18	330	0.01212121	0.18181818	0.19393939	0.000000000	0.012121212
19	613	0.12071778	0.10440457	0.22512235	0.011419250	0.003262643
20	979	0.07354443	0.13278856	0.20633299	0.004085802	0.008171604
	PositivityHE	SentimentLM	NegativityLM	PositivityLM	RatioUncertaintyLM	
1	0.000000000	0.007633588	0.000000000	0.007633588	0.007633588	
2	0.014134276	0.000000000	0.017667845	0.017667845	0.003533569	
3	0.006269592	-0.015673981	0.021943574	0.006269592	0.006269592	
4	0.015625000	0.023437500	0.007812500	0.031250000	0.007812500	
5	0.007653061	-0.015306122	0.033163265	0.017857143	0.007653061	
6	0.012500000	-0.043750000	0.065000000	0.021250000	0.015000000	
7	0.032348805	-0.014064698	0.036568214	0.022503516	0.015471167	
8	0.000000000	-0.207729469	0.207729469	0.000000000	0.009661836	
9	0.000000000	-0.006410256	0.032051282	0.025641026	0.006410256	
10	0.010416667	-0.024305556	0.045138889	0.020833333	0.000000000	
11	0.009259259	-0.012345679	0.021604938	0.009259259	0.012345679	
12	0.013059701	0.007462687	0.013059701	0.020522388	0.011194030	

13	0.007246377	0.021739130	0.004830918	0.026570048	0.000000000
14	0.007832898	-0.015665796	0.018276762	0.002610966	0.005221932
15	0.014884980	-0.005412720	0.013531800	0.008119080	0.004059540
16	0.012939002	-0.027726433	0.059149723	0.031423290	0.011090573
17	0.006191950	-0.006191950	0.026315789	0.020123839	0.010835913
18	0.012121212	-0.057575758	0.084848485	0.027272727	0.012121212
19	0.014681892	0.001631321	0.022838499	0.024469821	0.013050571
20	0.012257406	-0.026557712	0.044943820	0.018386108	0.013278856

SentimentQDAP NegativityQDAP PositivityQDAP

	SentimentQDAP	NegativityQDAP	PositivityQDAP
1	0.061068702	0.007633588	0.06870229
2	0.067137809	0.028268551	0.09540636
3	0.037617555	0.053291536	0.09090909
4	0.062500000	0.031250000	0.09375000
5	0.007653061	0.068877551	0.07653061
6	0.040000000	0.070000000	0.11000000
7	0.015471167	0.059071730	0.07454290
8	-0.043478261	0.120772947	0.07729469
9	0.153846154	0.032051282	0.18589744
10	0.017361111	0.083333333	0.10069444
11	0.013888889	0.069444444	0.08333333
12	0.080223881	0.063432836	0.14365672
13	0.144927536	0.014492754	0.15942029
14	0.020887728	0.044386423	0.06527415
15	0.078484438	0.056833559	0.13531800
16	-0.009242144	0.114602588	0.10536044
17	0.007739938	0.111455108	0.11919505
18	-0.027272727	0.169696970	0.14242424
19	0.115823817	0.068515498	0.18433931
20	0.057201226	0.089887640	0.14708887

```
# Add genre labels
SentimentA$genre <- genre_labels

# Mean by genre
cat("\nMean SentimentGI by genre:\n")
```

Mean SentimentGI by genre:

```
print(aggregate(SentimentGI ~ genre, data = SentimentA, FUN = mean))
```

genre	SentimentGI
1 Music	0.06197146
2 News	0.06075931
3 Recipe	0.05696566
4 Review	0.05644696

```
cat("\nMean PositivityGI by genre:\n")
```

Mean PositivityGI by genre:

```
print(aggregate(PositivityGI ~ genre, data = SentimentA, FUN = mean))
```

```
genre PositivityGI
1 Music    0.1218076
2 News    0.1804524
3 Recipe   0.1387697
4 Review   0.1901282
```

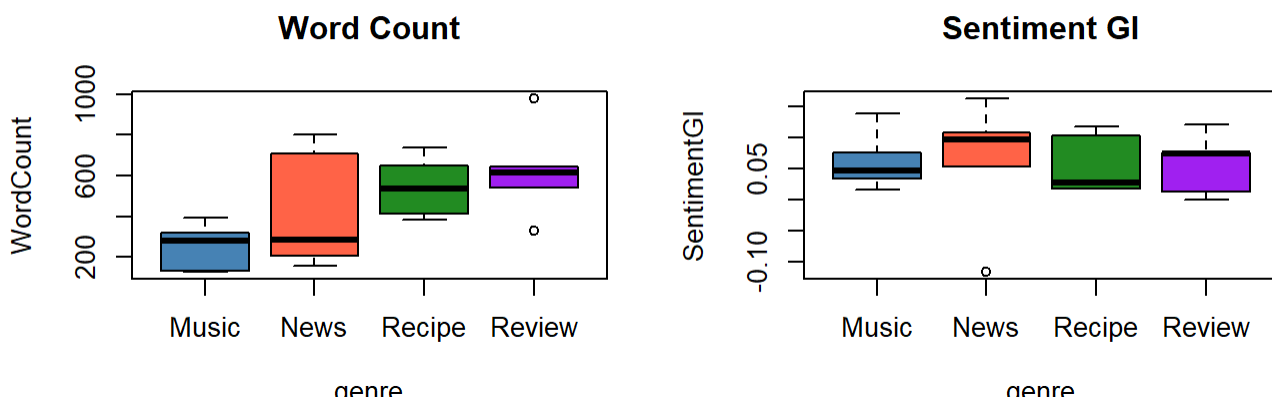
```
# SD by genre
cat("\nSD SentimentGI by genre:\n")
```

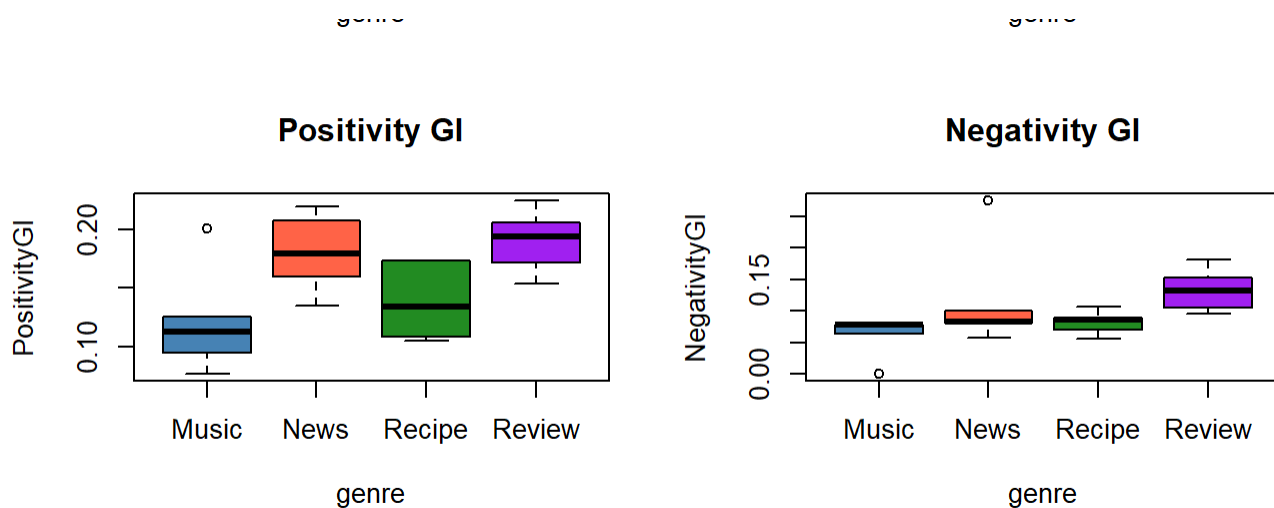
SD SentimentGI by genre:

```
print(aggregate(SentimentGI ~ genre, data = SentimentA, FUN = sd))
```

```
genre SentimentGI
1 Music    0.04785125
2 News    0.10615847
3 Recipe   0.04929931
4 Review   0.04987641
```

```
# Boxplots (4 measures like lecture slide 24)
par(mfrow = c(2, 2))
boxplot(WordCount ~ genre, data = SentimentA, main = "Word Count",
        col = c("steelblue", "tomato", "forestgreen", "purple"), frame = TRUE)
boxplot(SentimentGI ~ genre, data = SentimentA, main = "Sentiment GI",
        col = c("steelblue", "tomato", "forestgreen", "purple"), frame = TRUE)
boxplot(PositivityGI ~ genre, data = SentimentA, main = "Positivity GI",
        col = c("steelblue", "tomato", "forestgreen", "purple"), frame = TRUE)
boxplot(NegativityGI ~ genre, data = SentimentA, main = "Negativity GI",
        col = c("steelblue", "tomato", "forestgreen", "purple"), frame = TRUE)
```





```
par(mfrow = c(1, 1))

# ANOVA: test if SentimentGI differs significantly across genres
anova_result <- aov(SentimentGI ~ genre, data = SentimentA)
cat("\nANOVA result (SentimentGI):\n")
```

ANOVA result (SentimentGI):

```
print(summary(anova_result))
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
genre           3  0.00011  0.000038    0.008  0.999
Residuals      16  0.07391  0.004619
```

```
kable(aggregate(SentimentGI ~ genre, data = SentimentA, FUN = mean),
      col.names = c("Genre", "Mean SentimentGI"),
      digits = 4,
      caption = "Table 5.1 - Mean Sentiment (GI Dictionary) by Genre")
```

Table 5.1 - Mean Sentiment (GI Dictionary) by Genre

Genre	Mean SentimentGI
Music	0.0620
News	0.0608
Recipe	0.0570
Review	0.0564

```
kable(aggregate(SentimentGI ~ genre, data = SentimentA, FUN = sd),
      col.names = c("Genre", "SD SentimentGI"),
```

```
digits = 4,
caption = "Table 5.2 - Sentiment Variability by Genre")
```

Table 5.2 - Sentiment Variability by Genre

Genre	SD SentimentGI
Music	0.0479
News	0.1062
Recipe	0.0493
Review	0.0499

Task 6

```
# Helper: rescale vector to a range
rescale <- function(x, to = c(0, 1)) {
  if (max(x) == min(x)) return(rep(mean(to), length(x)))
  (x - min(x)) / (max(x) - min(x)) * (to[2] - to[1]) + to[1]
}

# Binary DTM
dtm_binary <- as.matrix(dtm_sparse)
dtm_binary[dtm_binary > 0] <- 1

kable(dtm_binary,
      caption = "Table 6.1 - Binary Document-Term Matrix",
      booktabs = FALSE) |>
kable_styling(bootstrap_options = c("striped", "condensed", "bordered"),
              full_width = FALSE,
              font_size = 10)
```

Table 6.1 – Binary Document-Term Matrix

	allow	becom	come	feel	first	found	live	long	made	make	new	part	peopl	person	reason
music_01.txt	0	0	1	0	1	0	0	0	1	0	1	0	0	1	0
music_02.txt	0	0	0	0	1	1	1	1	0	1	1	0	0	1	0
music_03.txt	0	1	0	1	1	0	1	1	0	0	0	0	0	0	0
music_04.txt	0	0	0	1	0	0	0	1	0	1	0	0	1	0	0
music_05.txt	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0
news_01.txt	1	0	0	0	0	0	0	0	0	0	1	1	0	1	0
news_02.txt	1	0	1	0	1	1	0	1	1	1	0	1	1	0	1
news_03.txt	0	0	0	0	1	1	0	0	1	1	0	0	0	0	0

news_04.txt	0	0	0	0	1	0	0	0	0	0	1	0	0	1	0
news_05.txt	1	0	0	0	1	1	0	0	0	1	0	0	0	0	0
recipe_01.txt	0	0	1	0	1	0	0	0	1	1	0	1	1	1	1
recipe_02.txt	0	1	0	0	0	0	0	0	1	1	0	1	1	0	1
recipe_03.txt	1	0	1	0	0	1	0	0	1	1	0	0	0	0	0
recipe_04.txt	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0
recipe_05.txt	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1
review_01.txt	1	1	0	1	1	0	1	1	1	1	1	1	1	0	0
review_02.txt	1	1	0	1	1	1	1	0	0	1	1	0	0	0	1
review_03.txt	0	1	1	1	1	1	1	1	0	1	1	0	1	1	1
review_04.txt	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1
review_05.txt	1	1	1	1	1	1	1	0	0	1	0	1	1	1	1

```
# Shared Term Matrix
doc_adj <- dtm_binary %*% t(dtm_binary)
diag(doc_adj) <- 0

kable(doc_adj,
      caption = "Table 6.2 - Shared Term Matrix (Document Adjacency)",
      booktabs = FALSE) |>
  kable_styling(bootstrap_options = c("striped", "condensed", "bordered"),
                full_width = FALSE,
                font_size = 10)
```

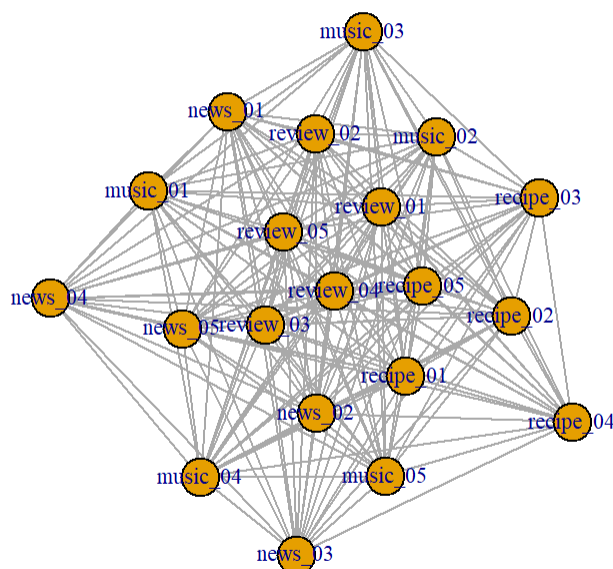
	music_01.txt	music_02.txt	music_03.txt	music_04.txt	music_05.txt	news_01.txt	news_02.txt	ne
music_01.txt	0	5	4	2	5	5	5	
music_02.txt	5	0	6	4	6	4	5	
music_03.txt	4	6	0	3	3	4	3	
music_04.txt	2	4	3	0	6	3	5	
music_05.txt	5	6	3	6	0	6	7	
news_01.txt	5	4	4	3	6	0	5	
news_02.txt	5	5	3	5	7	5	0	
news_03.txt	4	4	2	3	4	2	5	
news_04.txt	4	3	1	1	3	4	3	
news_05.txt	5	5	5	3	5	6	7	
recipe_01.txt	6	7	4	5	8	7	10	
recipe_02.txt	2	4	4	2	5	4	7	

recipe_03.txt	2	4	2	2	3	4	6
recipe_04.txt	1	4	3	2	3	2	4
recipe_05.txt	4	9	7	6	7	6	11
review_01.txt	5	8	8	6	9	7	8
review_02.txt	4	7	6	3	5	5	8
review_03.txt	7	8	8	5	6	6	10
review_04.txt	9	10	9	7	10	10	12
review_05.txt	5	9	7	7	7	9	11

```
# Build graph
doc_graph_basic <- graph_from_adjacency_matrix(
  doc_adj,
  mode      = "undirected",
  weighted = TRUE,
  diag      = FALSE
)

# Basic plot
set.seed(42)
plot(doc_graph_basic,
     vertex.label      = gsub("\\.txt", "", V(doc_graph_basic)$name),
     vertex.label.cex = 0.7,
     main              = "Figure 6.1 – Document Network (Basic)")
```

Figure 6.1 – Document Network (Basic)



```
# Improved plot
doc_graph <- doc_graph_basic

# Genre colours
genre_col <- c(
  Music = "#FFD700", # gold
  News = "#FF4500", # orange-red
  Recipe = "#00CC44", # green
  Review = "#9933FF" # purple
)

V(doc_graph)$genre <- genre_labels
V(doc_graph)$color <- genre_col[V(doc_graph)$genre]

# Degree centrality → node size
deg <- degree(doc_graph)

V(doc_graph)$size <- rescale(
  deg,
  to = c(12, 35)
)

# Edge width = similarity
E(doc_graph)$width <- rescale(
  E(doc_graph)$weight,
  to = c(0.5, 6)
)

# Edge transparency
edge_alpha <- rescale(
  E(doc_graph)$weight,
  to = c(0.15, 0.85)
)

E(doc_graph)$color <- rgb(
  0.4, 0.4, 0.4,
  alpha = edge_alpha
)

# Sentiment border colour
sent_scores <- SentimentA$SentimentGI

sent_norm <- rescale(
  sent_scores,
  to = c(0, 1)
```

```

    ~ ~ ~ ~ ~
)

sent_pal <- colorRamp(
  c("red", "white", "dodgerblue")
)

sent_rgb <- sent_pal(sent_norm)

V(doc_graph)$frame.color <- rgb(
  sent_rgb[,1],
  sent_rgb[,2],
  sent_rgb[,3],
  maxColorValue = 255
)

# Community detection
set.seed(42)

community <- cluster_louvain(doc_graph)

V(doc_graph)$community <- membership(community)

# Layout
set.seed(42)

layout_fr <- layout_with_fr(
  doc_graph,
  niter = 2000
)

# Plot with community hulls using mark.groups (cleaner)
# Get community membership
comm_members <- membership(community)
n_comm <- length(unique(comm_members))

# One hull colour per community
hull_fills <- adjustcolor(c("cyan", "pink", "lightgreen", "lightyellow"), alpha.f = 0.25)
hull_borders <- adjustcolor(c("cyan", "pink", "lightgreen", "lightyellow"), alpha.f = 0.60)

# Build mark.groups list
mark_groups <- lapply(sort(unique(comm_members)), function(i) {
  which(comm_members == i)
})

par(mar = c(2, 1, 4, 10))

plot(
  doc_graph,
  layout = layout_fr

```

```

layout      = layout_11,
mark.groups = mark_groups,
mark.col    = hull_fills[1:n_comm],
mark.border = hull_borders[1:n_comm],
vertex.label = gsub("\\.txt", "", V(doc_graph)$name),
vertex.label.cex = 0.70,
vertex.label.font = 2,
vertex.label.color = "black",
vertex.size = V(doc_graph)$size,
vertex.color = V(doc_graph)$color,
vertex.frame.color = V(doc_graph)$frame.color,
vertex.frame.width = 3,
edge.width = E(doc_graph)$width,
edge.color = E(doc_graph)$color,
edge.curved = 0.25,
main       = "Figure 6.2 - Document Network (Improved)\nNode size = degree centrality
)

# Genre legend
legend(
  x      = par("usr")[2] + 0.05,
  y      = par("usr")[4],
  legend = names(genre_col),
  fill   = genre_col,
  title  = "Genre (fill)",
  cex    = 0.8,
  bty    = "n",
  xpd    = TRUE
)

# Sentiment legend
legend(
  x      = par("usr")[2] + 0.05,
  y      = par("usr")[4] - 1,
  legend = c("Positive", "Neutral", "Negative"),
  fill   = c("dodgerblue", "white", "red"),
  border = "grey50",
  title  = "Sentiment GI\n(border colour)",
  cex    = 0.8,
  bty    = "n",
  xpd    = TRUE
)

```

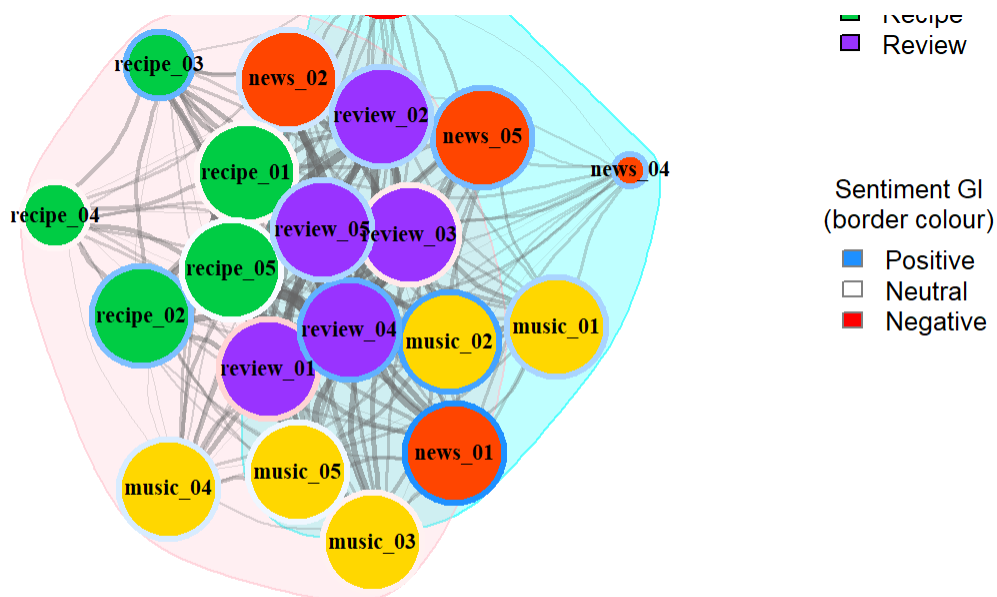
Figure 6.2 – Document Network (Improved)

= degree centrality | Edge width = similarity | Border = sentiment



Genre (fill)

- Music
- News
- Politics



```
par(mar = c(5, 4, 4, 2))

# Centrality table
centrality_df <- data.frame(
  Document    = gsub("\\.txt", "", V(doc_graph)$name),
  Genre       = V(doc_graph)$genre,
  Degree      = degree(doc_graph),
  Closeness   = round(closeness(doc_graph, normalized = TRUE), 4),
  Betweenness = round(betweenness(doc_graph, normalized = TRUE), 4),
  Eigenvector = round(eigen_centrality(doc_graph)$vector, 4),
  Sentiment   = round(sent_scores, 4),
  stringsAsFactors = FALSE
)

centrality_df <- centrality_df[order(-centrality_df$Degree), ]

kable(centrality_df,
  caption = "Table 6.3 – Document Centrality Measures (sorted by Degree)",
  row.names = FALSE,
  col.names = c("Document", "Genre", "Degree",
    "Closeness (norm.)", "Betweenness (norm.)",
    "Eigenvector", "Sentiment GI")) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE) |>
  row_spec(0, bold = TRUE) |>
  column_spec(3, bold = TRUE)
```

Table 6.3 – Document Centrality Measures (sorted by Degree)

Document	Genre	Degree	Closeness (norm.)	Betweenness (norm.)	Eigenvector	Sentiment GI
music_01	Music	19	0.2639	0.0239	0.4385	0.0763
music_02	Music	19	0.1881	0.0000	0.5878	0.1378
music_03	Music	19	0.2794	0.0302	0.4825	0.0157
music_04	Music	19	0.2969	0.0459	0.4068	0.0469
music_05	Music	19	0.2000	0.0000	0.5616	0.0332
news_01	News	19	0.2184	0.0000	0.5233	0.1625
news_02	News	19	0.1827	0.0000	0.6965	0.0534
news_03	News	19	0.2714	0.0136	0.3749	-0.1159
news_05	News	19	0.2135	0.0000	0.5328	0.1076
recipe_01	Recipe	19	0.1545	0.0000	0.8145	0.0185
recipe_02	Recipe	19	0.2676	0.0041	0.6009	0.1026
recipe_05	Recipe	19	0.1759	0.0000	0.9164	0.0271
review_01	Review	19	0.1776	0.0000	0.8136	0.0000
review_02	Review	19	0.1792	0.0000	0.7102	0.0759
review_03	Review	19	0.1532	0.0000	0.8379	0.0121
review_04	Review	19	0.1407	0.0000	1.0000	0.1207
review_05	Review	19	0.1597	0.0000	0.9146	0.0735
recipe_03	Recipe	18	0.2375	0.0012	0.4881	0.1184
recipe_04	Recipe	18	0.3016	0.1714	0.3641	0.0183
news_04	News	17	0.3276	0.3614	0.2857	0.0962

```
# Top 5 rankings table
```

```
top_n <- 5
```

```
rankings_table <- data.frame(
  Rank = 1:top_n,
  `Ranked by Betweenness` = centrality_df$Document[order(-centrality_df$Betweenness)][1:top_n],
  `Ranked by Closeness` = centrality_df$Document[order(-centrality_df$Closeness)][1:top_n],
  `Ranked by Eigenvector` = centrality_df$Document[order(-centrality_df$Eigenvector)][1:top_n],
  check.names = FALSE,
  stringsAsFactors = FALSE
)

kable(rankings_table,
  caption = "Table 6.4 - Top 5 Documents by Centrality Measure",
  row.names = FALSE) |>
kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
  full_width = FALSE) |>
row_spec(0, bold = TRUE)
```

Table 6.4 – Top 5 Documents by Centrality Measure

Rank	Ranked by Betweenness	Ranked by Closeness	Ranked by Eigenvector
1	news_04	news_04	review_04
2	recipe_04	recipe_04	recipe_05
3	music_04	music_04	review_05
4	music_03	music_03	review_03
5	music_01	news_03	recipe_01

```
# Community membership table ---
community_df <- data.frame(
  Document = gsub("\\.txt", "", names(membership(community))),
  Genre     = genre_labels,
  Community = membership(community),
  Degree    = degree(doc_graph)[names(membership(community))],
  Sentiment = round(sent_scores, 4),
  stringsAsFactors = FALSE
)
community_df <- community_df[order(community_df$Community, community_df$Genre), ]

kable(community_df,
  caption = "Table 6.5 – Community Membership with Sentiment",
  row.names = FALSE,
  col.names = c("Document", "Genre", "Community", "Degree", "Sentiment GI")) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE) |>
  row_spec(0, bold = TRUE)
```

Table 6.5 – Community Membership with Sentiment

Document	Genre	Community	Degree	Sentiment GI
music_01	Music	1	19	0.0763
music_05	Music	1	19	0.0332
news_01	News	1	19	0.1625
news_03	News	1	19	-0.1159
news_04	News	1	17	0.0962
news_05	News	1	19	0.1076
music_02	Music	2	19	0.1378
music_03	Music	2	19	0.0157
music_04	Music	2	19	0.0469
news_02	News	2	19	0.0534
recipe_01	Recipe	3	10	0.0105

recipe_01	Recipe	2	19	0.0185
recipe_02	Recipe	2	19	0.1026
recipe_03	Recipe	2	18	0.1184
recipe_04	Recipe	2	18	0.0183
recipe_05	Recipe	2	19	0.0271
review_01	Review	2	19	0.0000
review_02	Review	2	19	0.0759
review_03	Review	2	19	0.0121
review_04	Review	2	19	0.1207
review_05	Review	2	19	0.0735

```
# Confusion table + accuracy
conf_table <- table(Genre = community_df$Genre,
                   Community = community_df$Community)

kable(as.data.frame.matrix(conf_table),
      caption = "Table 6.6 – Community Detection vs Genre (Confusion Matrix)" |>
      kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
                   full_width = FALSE) |>
      add_header_above(c(" " = 1, "Detected Community" = ncol(conf_table))) |>
      row_spec(0, bold = TRUE)
```

Table 6.6 – Community Detection vs Genre (Confusion Matrix)

	Detected Community	
	1	2
Music	2	3
News	4	1
Recipe	0	5
Review	0	5

```
n_correct <- sum(apply(conf_table, 2, max))
accuracy <- n_correct / nrow(community_df)
cat(sprintf("Community Detection Accuracy (best-match): %.4f (%.1f%%)\n",
          accuracy, accuracy * 100))
```

Community Detection Accuracy (best-match): 0.4500 (45.0%)

```
# Isolated documents
isolated <- V(doc_graph)$name[degree(doc_graph) == 0]
if (length(isolated) > 0) {
  kable(data.frame(Isolated_Documents = gsub("\\.txt", "", isolated)),
```

```

      caption = "Table 6.7 - Isolated Documents (no shared terms)" |>
      kable_styling(bootstrap_options = c("striped"), full_width = FALSE)
    } else {
      cat("No isolated documents - all documents share at least one term.\n")
    }
  }

```

No isolated documents – all documents share at least one term.

Task 7

```

# Transpose DTM to get Term-Document Matrix
# Columns = documents, Rows = tokens
# Binary version: 1 if token appears in document, 0 otherwise
tdm_binary <- t(dtm_binary) # reuse binary DTM from Task 6

# Token Co-occurrence Matrix (Shared Term Matrix)
# Multiply TDM by its transpose → 26x26 adjacency matrix
# Each cell = number of documents both tokens co-occur in
token_adj <- tdm_binary %%% t(tdm_binary)
diag(token_adj) <- 0 # remove self-connections

kable(token_adj,
      caption = "Table 7.1 - Token Co-occurrence Matrix",
      booktabs = FALSE) |>
  kable_styling(bootstrap_options = c("striped", "condensed", "bordered"),
    full_width = FALSE,
    font_size = 10)

```

Table 7.1 – Token Co-occurrence Matrix

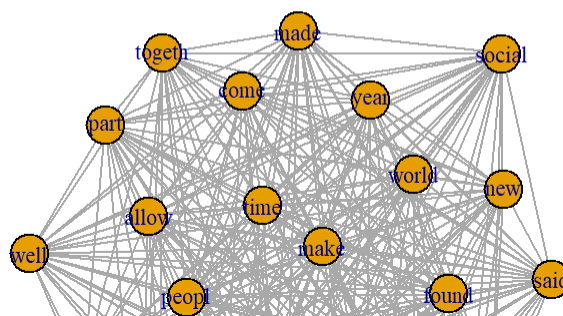
	allow	becom	come	feel	first	found	live	long	made	make	new	part	peopl	person	reason	said	s
allow	0	5	5	5	7	6	5	4	5	8	5	5	5	3	5	5	
becom	5	0	4	7	7	4	7	5	4	7	5	4	6	3	6	3	
come	5	4	0	4	7	5	4	4	6	7	4	4	6	5	6	4	
feel	5	7	4	0	7	4	7	6	3	7	5	3	6	3	5	4	
first	7	7	7	7	0	8	8	7	7	11	8	5	7	7	7	7	
found	6	4	5	4	8	0	5	4	4	9	4	2	4	3	5	4	
live	5	7	4	7	8	5	0	6	3	7	6	3	5	4	5	4	
long	4	5	4	6	7	4	6	0	6	9	6	4	6	3	4	6	
made	5	4	6	3	7	4	3	6	0	10	5	6	6	3	5	4	
make	8	7	7	7	11	9	7	9	10	0	7	7	9	5	8	7	
new	5	5	4	5	8	4	6	6	5	7	0	4	4	6	4	5	
part	5	4	4	3	5	2	3	4	6	7	4	0	6	4	5	5	

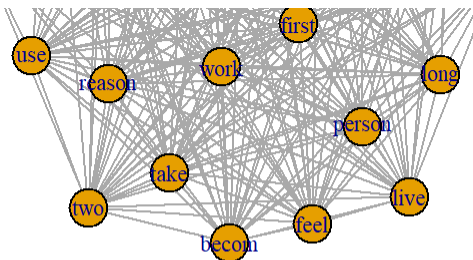
part	3	4	4	3	3	2	3	4	6	7	4	0	6	4	3	3
peopl	5	6	6	6	7	4	5	6	6	9	4	6	0	4	7	4
person	3	3	5	3	7	3	4	3	3	5	6	4	4	0	4	5
reason	5	6	6	5	7	5	5	4	5	8	4	5	7	4	0	3
said	5	3	4	4	7	4	4	6	4	7	5	5	4	5	3	0
social	4	4	3	4	7	3	4	4	4	5	5	3	3	4	2	5
take	6	4	5	5	5	3	4	5	6	8	5	6	6	4	4	5
time	8	7	6	6	10	6	6	6	7	11	8	8	8	6	8	6
togeth	4	4	5	3	5	4	4	4	6	8	3	4	5	4	5	3
two	3	3	3	4	6	4	4	5	5	8	4	4	5	3	3	4
use	5	5	5	4	6	5	4	4	5	8	3	4	5	4	5	4
well	4	4	5	3	5	5	3	4	6	8	3	3	5	2	6	1
work	7	7	5	6	8	5	7	6	7	10	6	6	6	5	6	5
world	4	6	4	5	9	4	6	6	6	9	7	5	5	5	5	6
year	6	4	6	5	10	6	4	5	6	10	7	6	6	7	6	8

```
# Build basic token graph
token_graph_basic <- graph_from_adjacency_matrix(
  token_adj,
  mode      = "undirected",
  weighted = TRUE,
  diag      = FALSE
)

# Basic plot
set.seed(42)
plot(token_graph_basic,
     vertex.label      = V(token_graph_basic)$name,
     vertex.label.cex = 0.7,
     main              = "Figure 7.1 - Token Network (Basic)")
```

Figure 7.1 – Token Network (Basic)





```
# Improved Token Network
token_graph <- token_graph_basic

# Node size = degree centrality (how many other tokens
# this token co-occurs with)
tok_deg <- degree(token_graph)

V(token_graph)$size <- rescale(tok_deg, to = c(8, 30))

# Node colour = eigenvector centrality band
tok_eig <- eigen_centrality(token_graph)$vector

eig_norm <- rescale(tok_eig, to = c(0, 1))
eig_pal <- colorRamp(c("#d0e8ff", "#1a6bbf")) # light → deep blue
eig_rgb <- eig_pal(eig_norm)
V(token_graph)$color <- rgb(eig_rgb[,1], eig_rgb[,2], eig_rgb[,3],
                             maxColorValue = 255)

# Edge width + transparency = co-occurrence strength
E(token_graph)$width <- rescale(E(token_graph)$weight, to = c(0.3, 6))

edge_alpha_tok <- rescale(E(token_graph)$weight, to = c(0.10, 0.80))
E(token_graph)$color <- rgb(0.4, 0.4, 0.4, alpha = edge_alpha_tok)

# Node label size scales with degree
V(token_graph)$label.cex <- rescale(tok_deg, to = c(0.55, 1.0))

# Community detection (Louvain)
set.seed(42)
tok_community <- cluster_louvain(token_graph)
tok_membership <- membership(tok_community)
V(token_graph)$community <- tok_membership

# Hull colours
n_tok_comm <- length(unique(tok_membership))
```

```

tok_hull_fills <- adjustcolor(
  c("pink","cyan","lightgreen","lightyellow","lightsalmon"),
  alpha.f = 0.25)[1:n_tok_comm]
tok_hull_borders <- adjustcolor(
  c("pink","cyan","lightgreen","lightyellow","lightsalmon"),
  alpha.f = 0.60)[1:n_tok_comm]

tok_mark_groups <- lapply(sort(unique(tok_membership)), function(i) {
  which(tok_membership == i)
}))

# Layout
set.seed(42)
tok_layout <- layout_with_fr(token_graph, niter = 2000)

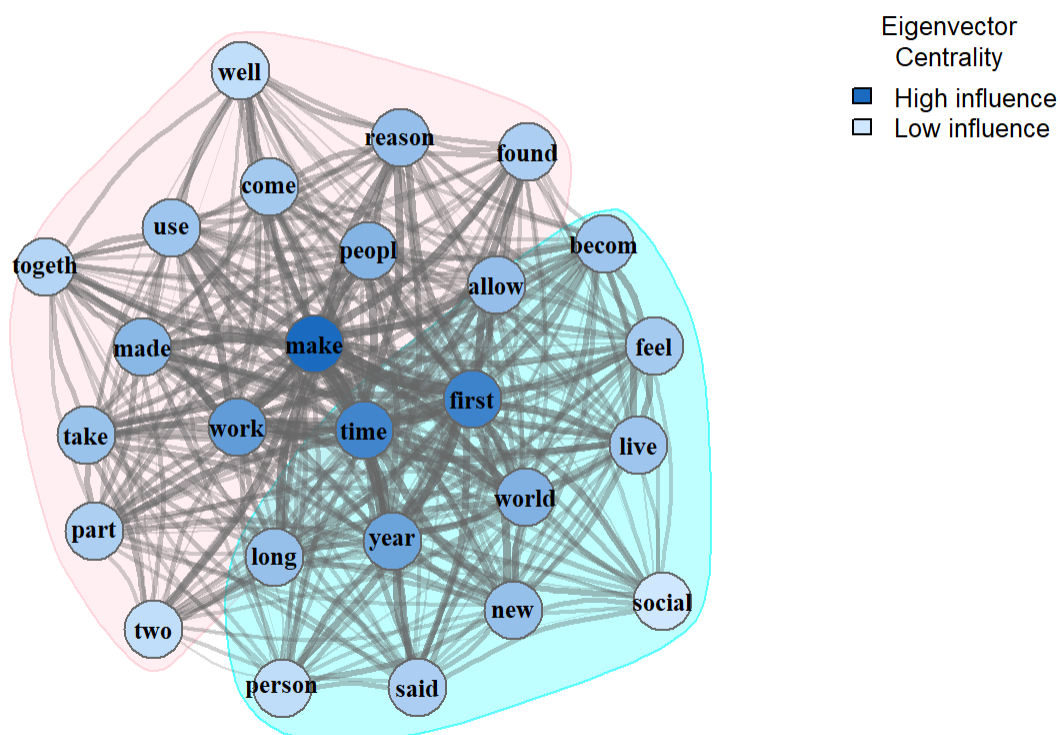
# Plot
par(mar = c(2, 1, 4, 10))

plot(
  token_graph,
  layout          = tok_layout,
  mark.groups     = tok_mark_groups,
  mark.col        = tok_hull_fills,
  mark.border     = tok_hull_borders,
  vertex.label    = V(token_graph)$name,
  vertex.label.cex = V(token_graph)$label.cex,
  vertex.label.font = 2,
  vertex.label.color = "black",
  vertex.size     = V(token_graph)$size,
  vertex.color    = V(token_graph)$color,
  vertex.frame.color = "grey40",
  edge.width      = E(token_graph)$width,
  edge.color      = E(token_graph)$color,
  edge.curved     = 0.2,
  main            = "Figure 7.2 - Token Network (Improved)"
)

# Eigenvector centrality colour legend
legend(
  x      = par("usr")[2] + 0.05,
  y      = par("usr")[4],
  legend = c("High influence", "Low influence"),
  fill   = c("#1a6bbf", "#d0e8ff"),
  title  = "Eigenvector\nCentrality",
  cex    = 0.8,
  bty    = "n",
  xpd    = TRUE
)

```

Figure 7.2 – Token Network (Improved)



```
par(mar = c(5, 4, 4, 2))

# Token Centrality Table
tok_centrality_df <- data.frame(
  Token      = V(token_graph)$name,
  Degree     = degree(token_graph),
  Closeness  = round(closeness(token_graph, normalized = TRUE), 4),
  Betweenness = round(betweenness(token_graph, normalized = TRUE), 4),
  Eigenvector = round(eigen centrality(token_graph)$vector, 4),
  Community  = tok_membership,
  stringsAsFactors = FALSE
)

tok_centrality_df <- tok_centrality_df[order(-tok_centrality_df$Degree), ]

kable(tok_centrality_df,
  caption  = "Table 7.2 – Token Centrality Measures (sorted by Degree)",
  row.names = FALSE,
  col.names = c("Token", "Degree", "Closeness (norm.)",
    "Betweenness (norm.)", "Eigenvector", "Community")) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE) |>
  row_sпец(0, hold = TRUE) |>
```

```

    row_spec(1, bold = TRUE),
    column_spec(2, bold = TRUE)

```

Table 7.2 – Token Centrality Measures (sorted by Degree)

Token	Degree	Closeness (norm.)	Betweenness (norm.)	Eigenvector	Community
allow	25	0.1938	0.0000	0.6579	1
becom	25	0.2016	0.0000	0.6361	2
come	25	0.2101	0.0000	0.6181	1
feel	25	0.2101	0.0008	0.6160	2
first	25	0.1420	0.0000	0.9005	2
found	25	0.2212	0.0064	0.5979	1
live	25	0.2016	0.0008	0.6346	2
long	25	0.1969	0.0000	0.6558	2
made	25	0.1908	0.0015	0.6872	1
make	25	0.1323	0.0000	1.0000	1
new	25	0.2000	0.0000	0.6567	2
part	25	0.2193	0.0076	0.5948	1
peopl	25	0.1866	0.0000	0.7038	1
person	25	0.2475	0.0199	0.5416	2
reason	25	0.2101	0.0000	0.6582	1
said	25	0.2451	0.0056	0.5977	2
social	25	0.2874	0.1219	0.4957	2
take	25	0.2033	0.0008	0.6432	1
time	25	0.1437	0.0000	0.8904	2
togeth	25	0.2604	0.0336	0.5722	1
two	25	0.2500	0.0051	0.5374	1
use	25	0.2033	0.0000	0.6346	1
well	25	0.2778	0.0733	0.5400	1
work	25	0.1613	0.0000	0.7992	1
world	25	0.1799	0.0000	0.7169	2
year	25	0.1773	0.0000	0.7737	2

```
# Top 5 rankings
```

```
top_n <- 5
```

```
tok_rankings <- data.frame(
```

```
  Rank = 1:top_n,
```

```

  Ranked by Betweenness = tok_centrality_df$Token[order(-tok_centrality_df$Betweenness)][1:top_5]
  `Ranked by Closeness` = tok_centrality_df$Token[order(-tok_centrality_df$Closeness)][1:top_5]
  `Ranked by Eigenvector` = tok_centrality_df$Token[order(-tok_centrality_df$Eigenvector)][1:top_5]
  check.names          = FALSE,
  stringsAsFactors     = FALSE
)

kable(tok_rankings,
      caption = "Table 7.3 – Top 5 Tokens by Centrality Measure",
      row.names = FALSE) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
                full_width = FALSE) |>
  row_spec(0, bold = TRUE)

```

Table 7.3 – Top 5 Tokens by Centrality Measure

Rank	Ranked by Betweenness	Ranked by Closeness	Ranked by Eigenvector
1	social	social	make
2	well	well	first
3	togeth	togeth	time
4	person	two	work
5	part	person	year

```

# Community membership table
tok_community_df <- data.frame(
  Token      = V(token_graph)$name,
  Community  = tok_membership,
  Degree     = degree(token_graph),
  Eigenvector = round(eigen Centrality(token_graph)$vector, 4),
  stringsAsFactors = FALSE
)
tok_community_df <- tok_community_df[order(tok_community_df$Community,
                                           -tok_community_df$Degree), ]

kable(tok_community_df,
      caption = "Table 7.4 – Token Community Membership",
      row.names = FALSE,
      col.names = c("Token", "Community", "Degree", "Eigenvector")) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
                full_width = FALSE) |>
  row_spec(0, bold = TRUE)

```

Table 7.4 – Token Community Membership

Token	Community	Degree	Eigenvector
allow	1	25	0.6579

come	1	25	0.6181
found	1	25	0.5979
made	1	25	0.6872
make	1	25	1.0000
part	1	25	0.5948
peopl	1	25	0.7038
reason	1	25	0.6582
take	1	25	0.6432
togeth	1	25	0.5722
two	1	25	0.5374
use	1	25	0.6346
well	1	25	0.5400
work	1	25	0.7992
becom	2	25	0.6361
feel	2	25	0.6160
first	2	25	0.9005
live	2	25	0.6346
long	2	25	0.6558
new	2	25	0.6567
person	2	25	0.5416
said	2	25	0.5977
social	2	25	0.4957
time	2	25	0.8904
world	2	25	0.7169
year	2	25	0.7737

```
# Isolated tokens check
isolated_tok <- V(token_graph)$name[degree(token_graph) == 0]
if (length(isolated_tok) > 0) {
  kable(data.frame(Isolated_Tokens = isolated_tok),
        caption = "Table 7.5 - Isolated Tokens") |>
  kable_styling(bootstrap_options = c("striped"), full_width = FALSE)
} else {
  cat("No isolated tokens – all tokens co-occur with at least one other token.\n")
}
```

No isolated tokens – all tokens co-occur with at least one other token.

Task 8

```
# Transform DTM into edge list format
# Each row = one document-token pair where token appears in document
dtm_matrix_full <- as.matrix(dtm_sparse)

# Convert to long-format data frame (document, token, weight)
bip_edges <- which(dtm_matrix_full > 0, arr.ind = TRUE)
bip_df <- data.frame(
  document = rownames(dtm_matrix_full)[bip_edges[, 1]],
  token     = colnames(dtm_matrix_full)[bip_edges[, 2]],
  weight    = dtm_matrix_full[bip_edges],
  stringsAsFactors = FALSE
)

# Clean document names
bip_df$document <- gsub("\\.txt", "", bip_df$document)

cat("Number of document-token edges:", nrow(bip_df), "\n")
```

Number of document-token edges: 254

```
cat("Unique documents:", length(unique(bip_df$document)), "\n")
```

Unique documents: 20

```
cat("Unique tokens:", length(unique(bip_df$token)), "\n")
```

Unique tokens: 26

```
# Build bipartite graph
bip_graph <- graph_from_data_frame(bip_df, directed = FALSE)

# Mark node types: TRUE = document, FALSE = token
V(bip_graph)$type <- V(bip_graph)$name %in% bip_df$document

cat("\nNode types:\n")
```

Node types:

```
cat("Document nodes:", sum(V(bip_graph)$type), "\n")
```

Document nodes: 20

```
cat("Token nodes:", sum(!V(bip_graph)$type), "\n")
```

Token nodes: 26

```
# Verify bipartite structure
cat("Is bipartite:", is_bipartite(bip_graph), "\n")
```

Is bipartite: TRUE

```
# Basic bipartite plot
set.seed(42)

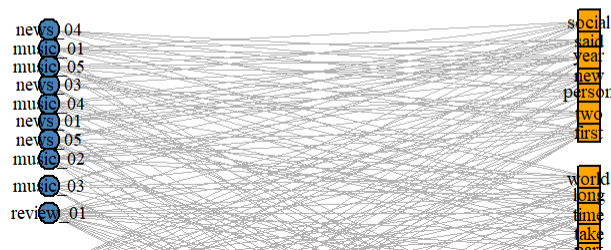
# Use bipartite layout
bip_layout <- layout_as_bipartite(bip_graph)

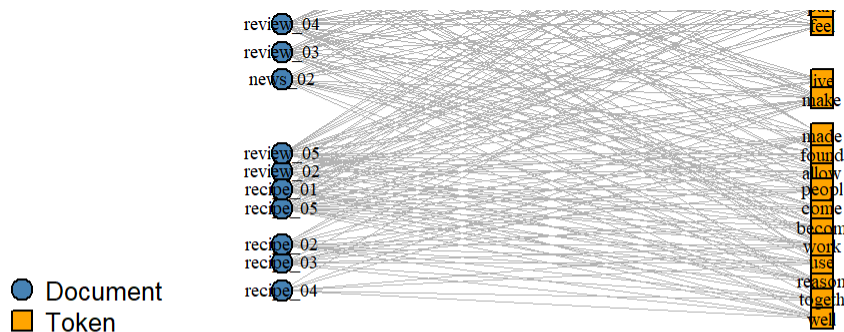
# Swap axes: documents on left, tokens on right (horizontal layout)
bip_layout_h <- bip_layout[, c(2, 1)]

plot(
  bip_graph,
  layout          = bip_layout_h,
  vertex.shape    = ifelse(V(bip_graph)$type, "circle", "square"),
  vertex.color    = ifelse(V(bip_graph)$type, "steelblue", "orange"),
  vertex.size     = 8,
  vertex.label.cex = 0.6,
  vertex.label.color = "black",
  edge.width      = 0.5,
  edge.color      = "grey70",
  main            = "Figure 8.1 – Bipartite Network (Basic)\nCircles = Documents | Squares = Tokens",
)

legend("bottomleft",
  legend = c("Document", "Token"),
  pch    = c(21, 22),
  pt.bg  = c("steelblue", "orange"),
  pt.cex = 1.5,
  cex    = 0.8,
  bty    = "n")
```

Figure 8.1 – Bipartite Network (Basic)
Circles = Documents | Squares = Tokens





```
# Improved Bipartite Plot
bip_imp <- bip_graph

# Node colour
# Documents → coloured by genre
# Tokens → coloured by degree (how many docs they appear in)

# Genre colours for document nodes (reuse from Task 6)
doc_names_clean <- gsub("\\.txt", "", names(docs))

V(bip_imp)$genre <- ifelse(
  V(bip_imp)$type,
  genre_labels[match(V(bip_imp)$name, doc_names_clean)],
  "Token"
)

genre_col_bip <- c(
  Music = "#FFD700",
  News = "#FF4500",
  Recipe = "#00CC44",
  Review = "#9933FF",
  Token = "#FF8C00" # orange for token nodes
)

V(bip_imp)$color <- genre_col_bip[V(bip_imp)$genre]

# Node size
# Documents → scaled by degree (number of tokens used)
# Tokens → scaled by degree (number of documents appeared in)
bip_deg <- degree(bip_imp)

V(bip_imp)$size <- rescale(bip_deg, to = c(6, 28))
```

```

# Node shape
# Documents = circle, Tokens = square
V(bip_imp)$shape <- ifelse(V(bip_imp)$type, "circle", "square")

# Node border colour for documents = sentiment
sent_norm_bip <- rescale(sent_scores, to = c(0, 1))
sent_pal_bip <- colorRamp(c("red", "white", "dodgerblue"))
sent_rgb_bip <- sent_pal_bip(sent_norm_bip)
sent_colors_bip <- rgb(sent_rgb_bip[,1], sent_rgb_bip[,2], sent_rgb_bip[,3],
                      maxColorValue = 255)

# Apply border colour: sentiment for docs, grey for tokens
V(bip_imp)$frame.color <- ifelse(
  V(bip_imp)$type,
  sent_colors_bip[match(V(bip_imp)$name, doc_names_clean)],
  "grey40"
)

# Edge width = term frequency weight
E(bip_imp)$width <- rescale(E(bip_imp)$weight, to = c(0.3, 4))

edge_alpha_bip <- rescale(E(bip_imp)$weight, to = c(0.15, 0.75))
E(bip_imp)$color <- rgb(0.4, 0.4, 0.4, alpha = edge_alpha_bip)

# Layout – Fruchterman-Reingold separates the two node
# types better than bipartite layout for dense graphs
set.seed(42)
bip_layout_fr <- layout_with_fr(bip_imp, niter = 3000)

# Plot
par(mar = c(2, 1, 4, 11))

plot(
  bip_imp,
  layout          = bip_layout_fr,
  vertex.shape    = V(bip_imp)$shape,
  vertex.size     = V(bip_imp)$size,
  vertex.color    = V(bip_imp)$color,
  vertex.frame.color = V(bip_imp)$frame.color,
  vertex.frame.width = 2.5,
  vertex.label    = V(bip_imp)$name,
  vertex.label.cex = ifelse(V(bip_imp)$type, 0.65, 0.60),
  vertex.label.font = 2,
  vertex.label.color = "black",
  edge.width      = E(bip_imp)$width,
  edge.color      = E(bip_imp)$color,
  edge.curved     = 0.15,
  main            = "Figure 8.2 – Bipartite Network (Improved)"
)

```

```

),

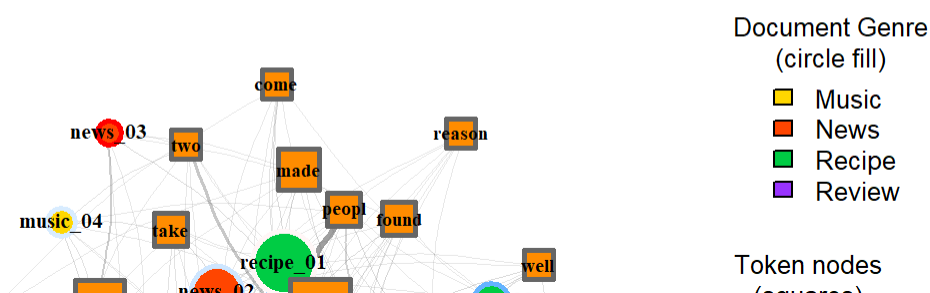
# Genre legend (document nodes)
legend(
  x      = par("usr")[2] + 0.05,
  y      = par("usr")[4],
  legend = c("Music", "News", "Recipe", "Review"),
  fill   = c("#FFD700", "#FF4500", "#00CC44", "#9933FF"),
  title  = "Document Genre\n(circle fill)",
  cex    = 0.8,
  bty    = "n",
  xpd    = TRUE
)

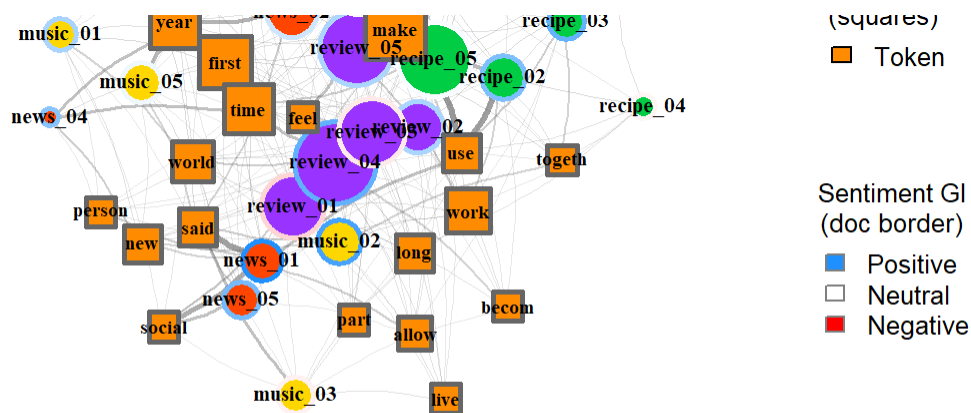
# Token legend
legend(
  x      = par("usr")[2] + 0.05,
  y      = par("usr")[4] - 0.8,
  legend = "Token",
  fill   = "#FF8C00",
  title  = "Token nodes\n(squares)",
  cex    = 0.8,
  bty    = "n",
  xpd    = TRUE
)

# Sentiment legend
legend(
  x      = par("usr")[2] + 0.05,
  y      = par("usr")[4] - 1.5,
  legend = c("Positive", "Neutral", "Negative"),
  fill   = c("dodgerblue", "white", "red"),
  border = "grey50",
  title  = "Sentiment GI\n(doc border)",
  cex    = 0.8,
  bty    = "n",
  xpd    = TRUE
)

```

Figure 8.2 – Bipartite Network (Improved)





```
par(mar = c(5, 4, 4, 2))

# Centrality tables
# Separate document and token nodes
doc_nodes <- V(bip_imp)$name[V(bip_imp)$type]
token_nodes <- V(bip_imp)$name[!V(bip_imp)$type]

# Document centrality
doc_centrality_df <- data.frame(
  Document = doc_nodes,
  Genre = genre_labels[match(doc_nodes, doc_names_clean)],
  Degree = degree(bip_imp)[doc_nodes],
  Closeness = round(closeness(bip_imp, normalized = TRUE)[doc_nodes], 4),
  Betweenness = round(betweenness(bip_imp, normalized = TRUE)[doc_nodes], 4),
  Sentiment = round(sent_scores[match(doc_nodes, doc_names_clean)], 4),
  stringsAsFactors = FALSE
)
doc_centrality_df <- doc_centrality_df[order(-doc_centrality_df$Degree), ]

kable(doc_centrality_df,
  caption = "Table 8.1 – Document Node Centrality in Bipartite Network",
  row.names = FALSE,
  col.names = c("Document", "Genre", "Degree",
    "Closeness (norm.)", "Betweenness (norm.)", "Sentiment GI")) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE) |>
  row_spec(0, bold = TRUE) |>
  column_spec(3, bold = TRUE)
```

Table 8.1 – Document Node Centrality in Bipartite Network

Document	Genre	Degree	Closeness (norm.)	Betweenness (norm.)	Sentiment GI
review_04	Review	22	0.4891	0.0785	0.1207

recipe_05	Recipe	20	0.4688	0.0737	0.0271
review_05	Review	20	0.4128	0.0284	0.0735
review_03	Review	18	0.5114	0.1463	0.0121
review_01	Review	17	0.4592	0.0358	0.0000
recipe_01	Recipe	17	0.4787	0.0790	0.0185
news_02	News	14	0.4455	0.0430	0.0534
review_02	Review	14	0.4245	0.0176	0.0759
recipe_02	Recipe	12	0.3846	0.0087	0.1026
music_02	Music	12	0.4206	0.0199	0.1378
news_01	News	11	0.3125	0.0020	0.1625
music_05	Music	11	0.4245	0.0293	0.0332
news_05	News	10	0.4054	0.0167	0.1076
recipe_03	Recipe	10	0.4206	0.0316	0.1184
music_03	Music	10	0.4167	0.0322	0.0157
music_01	Music	9	0.4054	0.0186	0.0763
music_04	Music	8	0.3750	0.0113	0.0469
news_03	News	7	0.3982	0.0140	-0.1159
recipe_04	Recipe	7	0.3659	0.0084	0.0183
news_04	News	5	0.3214	0.0000	0.0962

```
# Token centrality
tok_centrality_bip_df <- data.frame(
  Token      = token_nodes,
  Degree      = degree(bip_imp)[token_nodes],
  Closeness   = round(closeness(bip_imp, normalized = TRUE)[token_nodes], 4),
  Betweenness = round(betweenness(bip_imp, normalized = TRUE)[token_nodes], 4),
  stringsAsFactors = FALSE
)
tok_centrality_bip_df <- tok_centrality_bip_df[order(-tok_centrality_bip_df$Degree), ]

kable(tok_centrality_bip_df,
  caption   = "Table 8.2 - Token Node Centrality in Bipartite Network",
  row.names = FALSE,
  col.names = c("Token", "Degree (docs containing token)",
    "Closeness (norm.)", "Betweenness (norm.)") |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE) |>
  row_spec(0, bold = TRUE) |>
  column_spec(2, bold = TRUE)
```

Table 8.2 – Token Node Centrality in Bipartite Network

Token	Degree (docs containing token)	Closeness (norm.)	Betweenness (norm.)
make	16	0.4245	0.0102
first	14	0.4545	0.0346
time	13	0.4128	0.0132
year	13	0.3982	0.0269
work	12	0.3750	0.0083
made	11	0.4639	0.0567
world	11	0.4245	0.0209
long	10	0.4206	0.0209
new	10	0.4545	0.0179
said	10	0.3629	0.0049
use	10	0.4018	0.0080
allow	9	0.4369	0.0269
found	9	0.4245	0.0235
peopl	9	0.4545	0.0274
take	9	0.4455	0.0206
becom	8	0.4327	0.0186
come	8	0.4592	0.0250
feel	8	0.3750	0.0070
live	8	0.4167	0.0088
part	8	0.4245	0.0121
person	8	0.4412	0.0564
reason	8	0.4737	0.0467
social	8	0.4286	0.0225
togeth	8	0.3947	0.0092
two	8	0.4286	0.0213
well	8	0.4286	0.0103

```

# Which documents share the most tokens
# Project bipartite graph onto document side
bip_proj <- bipartite_projection(bip_imp, which = "true")

doc_shared <- as_data_frame(bip_proj, what = "edges")
doc_shared <- doc_shared[order(-doc_shared$weight), ]

kable(head(doc_shared, 10),
      caption = "Table 8.3 – Top 10 Document Pairs by Shared Tokens (Bipartite Projection)",

```

```

    row.names = FALSE,
    col.names = c("Document A", "Document B", "Shared Tokens")) |>
kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
              full_width = FALSE) |>
row_spec(0, bold = TRUE)

```

Table 8.3 – Top 10 Document Pairs by Shared Tokens (Bipartite Projection)

Document A	Document B	Shared Tokens
review_04	review_05	17
recipe_05	review_04	16
recipe_05	review_05	16
review_01	review_04	16
review_04	review_03	15
recipe_05	review_01	14
recipe_05	review_03	14
review_04	recipe_01	14
review_05	recipe_01	14
recipe_05	recipe_01	13

```

# Which tokens appear in the most documents
token_doc_count <- data.frame(
  Token      = token_nodes,
  NumDocs    = degree(bip_imp)[token_nodes],
  stringsAsFactors = FALSE
)
token_doc_count <- token_doc_count[order(-token_doc_count$NumDocs), ]

kable(token_doc_count,
      caption = "Table 8.4 – Tokens by Number of Documents (Degree in Bipartite Network)",
      row.names = FALSE,
      col.names = c("Token", "Number of Documents")) |>
kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
              full_width = FALSE) |>
row_spec(0, bold = TRUE)

```

Table 8.4 – Tokens by Number of Documents (Degree in Bipartite Network)

Token	Number of Documents
make	16
first	14
time	13

year	13
work	12
made	11
world	11
long	10
new	10
said	10
use	10
allow	9
found	9
peopl	9
take	9
becom	8
come	8
feel	8
live	8
part	8
person	8
reason	8
social	8
togeth	8
two	8
well	8